LABORATORIO CLÍNICO

Sistema de Gestión de Laboratorio Clínico

CÓDIGO FUENTE

Arquitectura, piezas centrales del sistema e índice de archivos

Versión 1.0 · Octubre de 2026

Código fuente:

https://github.com/julioticacala53-web/bioaxis-lab

Descarga del programa (Windows y Linux):

https://github.com/julioticacala53-web/bioaxis-lab/releases/latest

ÍNDICE

1. Presentación....3
1.1 Cómo leer este documento....3
1.2 Tecnologías....4
2. Arquitectura....5
2.1 Recorrido de una petición....5
2.2 Estructura del proyecto....5
2.3 Nombres del dominio....6
3. Arranque y configuración....7
4. Base de datos....12
5. Modelo de datos....15
6. Reglas centrales....22
7. Flujo de atención....25
8. Informe de resultados y exportaciones....32
9. Respaldo y eliminación de atenciones....43
10. Auditoría y responsable....50
11. API REST....52
12. Interfaz de usuario....61
13. Pruebas automáticas....68
14. Empaquetado....71
15. Índice completo de archivos....73

1. PRESENTACIÓN

Este documento presenta el código fuente del Sistema de Gestión de Laboratorio Clínico: cómo está organizado, qué tecnologías usa, cómo se relacionan sus partes y el código completo de sus piezas centrales. El código íntegro, con todos sus archivos, está publicado en el repositorio:

- https://github.com/julioticacala53-web/bioaxis-lab

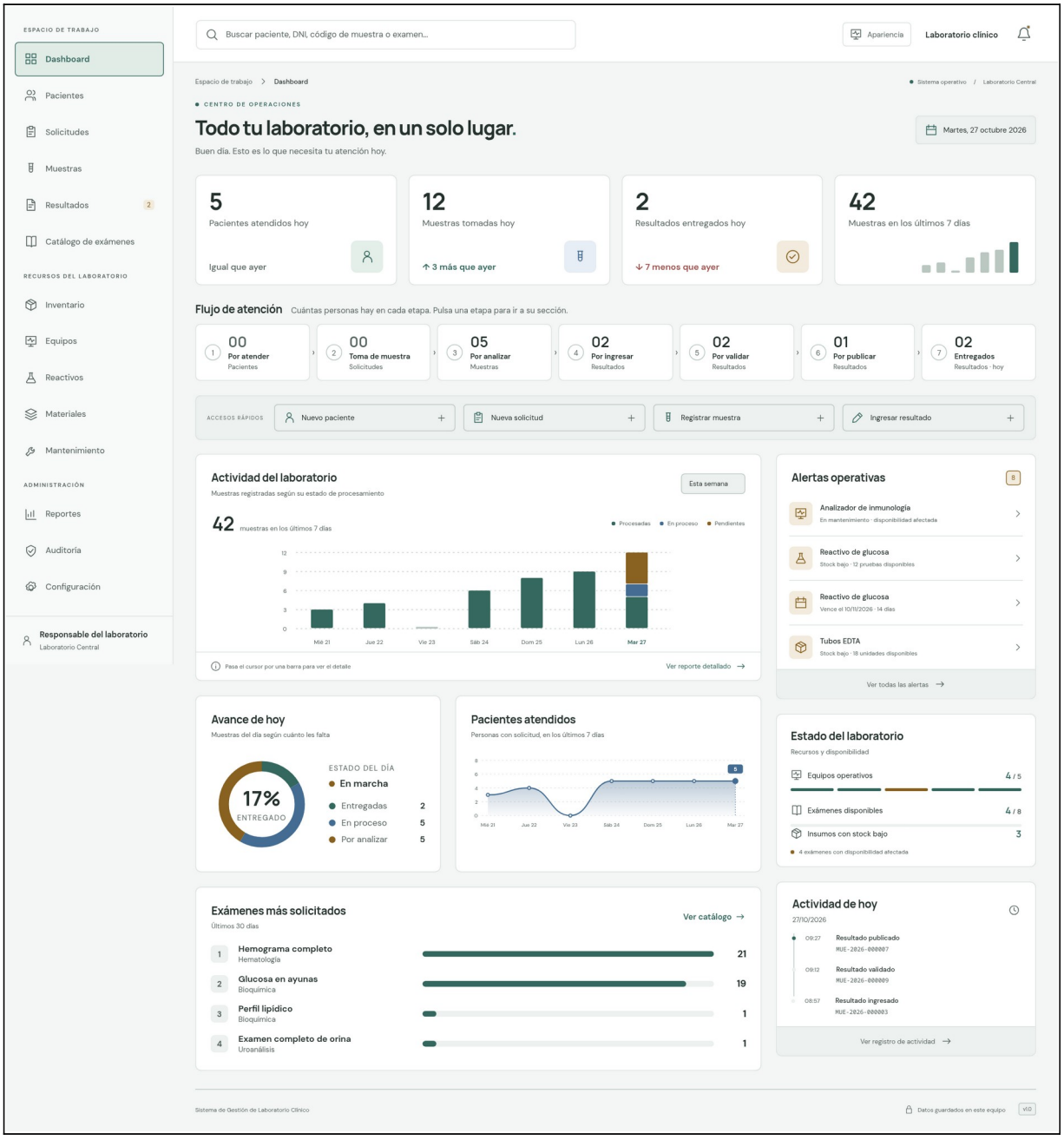


Figura 1. *Ventana principal del sistema*

1.1 Cómo leer este documento

- Cada archivo se presenta con su ruta, una explicación de lo que hace y su código en un recuadro titulado **CÓDIGO FUENTE**.

- En los recuadros el código se muestra **sin comentarios internos**, para que se lea limpio; la explicación de cada pieza está en el texto que lo precede.
- Cuando un archivo es muy extenso, se muestra solo el fragmento relevante y se indica así en su ruta.
- El proyecto tiene 159 archivos y unas 17300 líneas de código. El capítulo 15 los enumera todos con su función.

1.2 Tecnologías

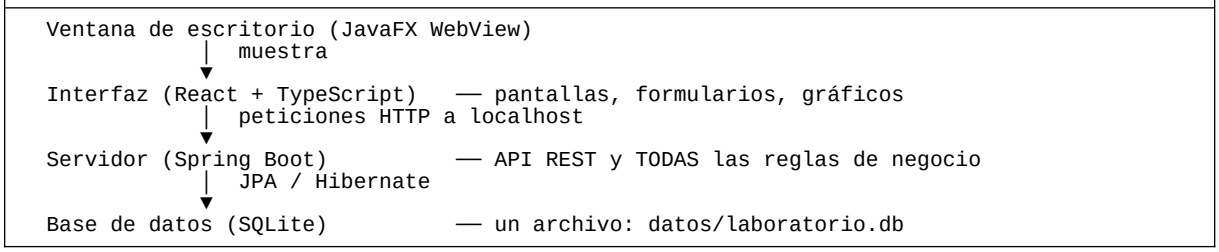
Tabla 1. *Tecnologías del sistema*

Tecnología	Versión	Uso
Java	21	Lenguaje del servidor y de la ventana de escritorio.
Spring Boot	3.3.4	Servidor de la aplicación y API REST.
Spring Data JPA / Hibernate	6.5	Acceso a la base de datos mediante entidades.
SQLite (sqlite-jdbc)	3.46.1	Base de datos en un único archivo.
JavaFX (WebView)	21.0.4	Ventana de escritorio que muestra la interfaz.
OpenPDF	2.0.3	Generación de documentos PDF.
Apache POI	5.3.0	Generación de hojas Excel (.xlsx).
React	19	Interfaz de usuario.
TypeScript	5.7	Lenguaje de la interfaz.
Vite	8	Compilación de la interfaz.
Apache Maven	3.9	Compilación, pruebas y empaquetado.
JUnit 5 / Spring Test	—	Pruebas automáticas (134).
jpackage / launch4j	21 / 3.50	Paquete de Linux / ejecutable de Windows.

2. ARQUITECTURA

El sistema es una aplicación de escritorio construida en tres capas que se ejecutan en el mismo equipo:

ESQUEMA



- **Ventana.** Al abrir el programa, VentanaVizcacha muestra la pantalla de carga, arranca el servidor en un puerto libre del propio equipo y, cuando está listo, carga la interfaz en un navegador interno (WebView).
- **Interfaz.** Dibuja las pantallas y envía cada acción al servidor. No contiene reglas de negocio: recibe los datos ya calculados (disponibilidad, estados, alertas).
- **Servidor.** Valida cada operación, aplica las reglas (disponibilidad, flujo de estados, descuentos de stock), registra la auditoría y devuelve la fotografía actualizada de los datos.
- **Base de datos.** Un único archivo SQLite junto al programa, por lo que los datos viajan con la carpeta.

2.1 Recorrido de una petición

Toda modificación sigue el mismo camino: la interfaz envía un POST a la API; el controlador lo pasa al servicio; el servicio valida, guarda mediante los repositorios y registra la auditoría; y la API responde con la fotografía completa de los datos (snapshot) más el código del registro creado en la cabecera X-Record-Id. La interfaz reemplaza sus datos por esa respuesta, de modo que nunca quedan desincronizados. Si una regla no se cumple, el servicio lanza BusinessException y la API responde 422 con el mensaje que la interfaz muestra al usuario.

2.2 Estructura del proyecto

ESTRUCTURA

```
(carpeta del proyecto)/
├── pom.xml                      proyecto Maven
├── README.md
├── src/main/java/pe/vizcacha/laboratorio/
│   ├── VizcachaApplication.java  aplicación Spring Boot
│   ├── DesktopLauncher.java      punto de entrada
│   ├── VentanaVizcacha.java      ventana de escritorio
│   ├── config/                   reloj y carga inicial
│   ├── domain/                   entidades, enumeraciones y conversores
│   ├── dto/                      datos de entrada y salida de la API
│   ├── repo/                     acceso a la base de datos
│   ├── service/                  reglas de negocio
│   └── web/                      controladores REST
├── src/main/resources/
│   ├── application.yml           configuración
│   └── db/schema.sql             estructura de la base de datos
└── src/test/java/...            pruebas automáticas
```

ESTRUCTURA

```
├── frontend/                  interfaz (React + TypeScript)
│   └── src/                   pantallas, componentes y estilos
└── packaging/                 scripts y recursos de empaquetado
```

2.3 Nombres del dominio

Las entidades del código llevan nombres de fauna andina. Es una convención interna que no afecta a lo que se ve en pantalla: los códigos visibles siguen siendo PAC-, SOL-, MUE-, etc.

Tabla 2. *Nombres de las entidades*

Concepto	Clase	Código en pantalla
Paciente	Puma	PAC-000001
Solicitud	Condor	SOL-000001
Muestra	Vicuna	MUE-2026-000001
Examen	Llama	EX-001
Recurso (equipo, reactivo, material)	Alpaca	EQ-, REA-, MAT-
Mantenimiento	Tapir	MAN-000001
Auditoría	Buho	AUD-000001

3. ARRANQUE Y CONFIGURACIÓN

DesktopLauncher.java

Ruta: src/main/java/pe/vizcacha/laboratorio/DesktopLauncher.java

Punto de entrada del ejecutable. Si el equipo tiene pantalla, abre la ventana de escritorio; con la propiedad laboratorio.sin-ventana o sin pantalla, arranca solo el servidor.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio;

import javafx.application.Application;
import org.springframework.boot.builder.SpringApplicationBuilder;

import java.awt.GraphicsEnvironment;

public final class DesktopLauncher {

    private static final String SIN_VENTANA = "laboratorio.sin-ventana";

    private DesktopLauncher() {
    }

    public static void main(String[] args) {
        VizcachaApplication.ensureDataDirectory();

        if (Boolean.getBoolean(SIN_VENTANA) || GraphicsEnvironment.isHeadless()) {
            new SpringApplicationBuilder(VizcachaApplication.class).run(args);
            return;
        }
        Application.launch(VentanaVizcacha.class, args);
    }
}
```

VizcachaApplication.java

Ruta: src/main/java/pe/vizcacha/laboratorio/VizcachaApplication.java

Aplicación Spring Boot. Antes de arrancar decide dónde viven los datos: junto al programa si la carpeta permite escritura (modo portátil) o en la carpeta personal del usuario (modo instalado), y crea las carpetas.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

@SpringBootApplication
public class VizcachaApplication {

    private static final String CARPETA_DATOS = "app.data-dir";
    private static final String CARPETA_EXPORTACIONES = "app.export-dir";

    public static void main(String[] args) {
        ensureDataDirectory();
        SpringApplication.run(VizcachaApplication.class, args);
    }
```

CÓDIGO FUENTE

```
    public static void ensureDataDirectory() {
        Path base = carpetaBase();
        crear(base.resolve("datos"), CARPETA_DATOS);

        if (System.getProperty(CARPETA_EXPORTACIONES) == null) {
            System.setProperty(CARPETA_EXPORTACIONES,
                    base.resolve("exportaciones").toString());
        }
    }

    private static Path carpetaBase() {
        String configurada = System.getProperty(CARPETA_DATOS);
        if (configurada != null && !configurada.isBlank()) {

            Path datos = Paths.get(configurada);
            crear(datos, null);
            return datos.getParent() == null ? Paths.get("") : datos.getParent();
        }

        Path trabajo = Paths.get("").toAbsolutePath();
        if (Files.isWritable(trabajo)) {
            return trabajo;
        }
        return Paths.get(System.getProperty("user.home"),
                ".local", "share", "Laboratorio");
    }

    private static void crear(Path carpeta, String propiedad) {
        try {
            Files.createDirectories(carpeta);
        } catch (IOException e) {
            throw new IllegalStateException(
                    "No se pudo crear la carpeta de datos: " + carpeta, e);
        }
        if (propiedad != null && System.getProperty(propiedad) == null) {
            System.setProperty(propiedad, carpeta.toString());
        }
    }
}
```

VentanaVizcacha.java

Ruta: src/main/java/pe/vizcacha/laboratorio/VentanaVizcacha.java

Ventana de escritorio. Muestra la pantalla de carga, arranca el servidor en otro hilo, y cuando responde carga la interfaz en el WebView. Ofrece a los diálogos del sistema la ventana principal como propietaria para que aparezcan siempre delante.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio;

import javafx.application.Platform;
import javafx.application.Application;
import javafx.concurrent.Worker;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.web.WebView;
import javafx.stage.Stage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.context.ConfigurableApplicationContext;
```

CÓDIGO FUENTE

```
public class VentanaVizcacha extends Application {

    private static final Logger log = LoggerFactory.getLogger(VentanaVizcacha.class);

    private static final String FONDO = "#bfc5c7";
    private static final String TURQUESA = "#273335";
    private static final String TENUE = "#425155";

    private ConfigurableApplicationContext spring;
    private Label mensaje;
    private StackPane raiz;
    private Stage ventana;
    private static volatile Stage principal;

    public static Stage principal() {
        return principal;
    }
    private VBox carga;

    @Override
    public void start(Stage ventana) {

        System.setProperty("laboratorio.ventana", "true");
        this.ventana = ventana;
        principal = ventana;
        ventana.setTitle("Laboratorio clínico — Sistema de Gestión de Laboratorio
Clínico");
        ventana.setMinWidth(1100);
        ventana.setMinHeight(700);

        carga = pantallaDeCarga();

        carga.setStyle("-fx-background-color: " + FONDO + ";");

        raiz = new StackPane(carga);
        raiz.setStyle("-fx-background-color: " + FONDO + ";");
        ventana.setScene(new Scene(raiz, 1440, 900, Color.web(FONDO)));
        ventana.setMaximized(true);
        ventana.show();

        Thread arranque = new Thread(this::levantarServicio, "arranque-laboratorio");
        arranque.setDaemon(true);
        arranque.start();
    }

    private VBox pantallaDeCarga() {
        Label marca = new Label("Laboratorio clínico");
        marca.setFont(Font.font("System", FontWeight.BOLD, 28));
        marca.setTextFill(Color.web(TURQUESA));

        Label sub = new Label("SISTEMA DE GESTIÓN DE LABORATORIO CLÍNICO");
        sub.setFont(Font.font("System", 11));
        sub.setTextFill(Color.web(TENUE));

        mensaje = new Label("Iniciando el laboratorio...");
        mensaje.setFont(Font.font("System", 12));
        mensaje.setTextFill(Color.web(TENUE));
        VBox.setMargin(mensaje, new Insets(26, 0, 0, 0));

        VBox caja = new VBox(8, marca, sub, mensaje);
        caja.setAlignment(Pos.CENTER);
        return caja;
    }

    private void levantarServicio() {
        try {

            spring = new SpringApplicationBuilder(VizcachaApplication.class)
                    .headless(false)
                    .logStartupInfo(false)
                    .run(getParameters().getRaw().toArray(new String[0]));
```

CÓDIGO FUENTE

```
            int puerto = spring.getEnvironment()
                    .getProperty("local.server.port", Integer.class, 0);

            log.info("Servicio en pie en el puerto {}", puerto);
            Platform.runLater(() -> mostrarInterfaz(puerto));
        } catch (Exception e) {
            Platform.runLater(() -> mensaje.setText(
                    "No se pudo iniciar el laboratorio: " + e.getMessage()));
        }
    }

    private void mostrarInterfaz(int puerto) {
        WebView navegador = new WebView();
        navegador.setContextMenuEnabled(false);
        navegador.setPageFill(Color.web(FONDO));

        raiz.getChildren().add(0, navegador);

        var motor = navegador.getEngine();
        motor.getLoadWorker().stateProperty().addListener((obs, antes, ahora) -> {

            log.info("Carga de la interfaz: {} -> {}", antes, ahora);
            if (ahora == Worker.State.FAILED || ahora == Worker.State.CANCELLED) {
                log.error("La carga fallo", motor.getLoadWorker().getException());
            }
            if (ahora == Worker.State.SUCCEEDED) {

                raiz.getChildren().remove(carga);
            } else if (ahora == Worker.State.FAILED) {
                mensaje.setText("No se pudo cargar la interfaz del laboratorio.");
            }
        });

        String url = "http://localhost:" + puerto + "/";
        log.info("Pidiendo la interfaz a {}", url);
        mensaje.setText("Cargando la interfaz...");
        motor.load(url);
    }

    @Override
    public void stop() {
        if (spring != null) {
            spring.close();
        }
        Platform.exit();
    }
}
```

src/main/resources/application.yml

Ruta: src/main/resources/application.yml

Configuración: base de datos SQLite con una sola conexión y espera ante bloqueos, puerto libre automático, carpeta de exportaciones y archivo de registro.

CÓDIGO FUENTE

```
spring:
  application:
    name: laboratorio-clinico
  datasource:
    url: jdbc:sqlite:${app.data-dir:datos}/laboratorio.db?
foreign_keys=on&journal_mode=WAL&busy_timeout=5000
    hikari:
      maximum-pool-size: 1
      pool-name: laboratorio-sqlite
    driver-class-name: org.sqlite.JDBC
  jpa:
    database-platform: org.hibernate.community.dialect.SQLiteDialect
    hibernate:
      ddl-auto: none
    open-in-view: false
```

CÓDIGO FUENTE

```
  sql:
    init:
      mode: always
      schema-locations: classpath:db/schema.sql
      continue-on-error: false

server:
  port: 0

app:
  demo-date: auto
  export-dir: exportaciones
  lab: Laboratorio clínico
  branch: Laboratorio Central

logging:
  level:
    org.hibernate.SQL: info
  file:
    name: ${app.data-dir:datos}/laboratorio.log
```

config/DemoClock.java

Ruta: src/main/java/pe/vizcacha/laboratorio/config/DemoClock.java

Reloj único del sistema. Todo el programa obtiene la fecha y la hora a través de esta clase, lo que permite fijarlas en las pruebas.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;

@Component
public class DemoClock {

    private static final String AUTOMATICO = "auto";

    private final LocalDate pinned;

    public DemoClock(@Value("${app.demo-date:auto}") String demoDate) {
        this.pinned = (demoDate == null || demoDate.isBlank()
                || AUTOMATICO.equalsIgnoreCase(demoDate.trim()))
                ? null
                : LocalDate.parse(demoDate.trim());
    }

    public LocalDate today() {
        return pinned != null ? pinned : LocalDate.now();
    }

    public String todayIso() {
        return today().toString();
    }

    public LocalDateTime timestamp() {
        return pinned != null
                ? LocalDateTime.of(pinned, LocalTime.now().withNano(0))
                : LocalDateTime.now().withNano(0);
    }
}
```

4. BASE DE DATOS

La estructura se crea al arrancar con sentencias CREATE TABLE IF NOT EXISTS, de modo que una base existente nunca se borra. Son trece tablas: pacientes, recursos, exámenes, solicitudes y sus exámenes, muestras con sus pasos de trazabilidad y sus resultados, mantenimientos, auditoría, usuarios (sin uso), configuración y preferencias.

db/schema.sql

Ruta: src/main/resources/db/schema.sql

Definición de tablas e índices.

CÓDIGO FUENTE

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS patient (
  id          TEXT PRIMARY KEY,
  dni         TEXT NOT NULL UNIQUE,
  first_name  TEXT NOT NULL,
  last_name   TEXT NOT NULL,
  birth       TEXT NOT NULL,
  sex         TEXT NOT NULL,
  phone       TEXT,
  email       TEXT,
  address     TEXT,
  notes       TEXT,
  created     TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS resource (
  id          TEXT PRIMARY KEY,
  kind        TEXT NOT NULL,
  name        TEXT NOT NULL,
  brand       TEXT, model TEXT, serial TEXT,
  area        TEXT NOT NULL,
  status      TEXT NOT NULL,

  qty         INTEGER NOT NULL DEFAULT 0,
  min_qty     INTEGER NOT NULL DEFAULT 0,
  unit        TEXT NOT NULL DEFAULT 'unidades',
  lot         TEXT,
  expires     TEXT,
  last_maint  TEXT, next_maint TEXT,
  responsible TEXT
);

CREATE TABLE IF NOT EXISTS exam (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  category    TEXT NOT NULL,
  price       REAL NOT NULL,
  time_label  TEXT NOT NULL,
  sample_type TEXT NOT NULL,
  volume      TEXT NOT NULL,
  prep        TEXT,
  equipment_id TEXT NOT NULL REFERENCES resource(id),
  reagent_id   TEXT NOT NULL REFERENCES resource(id),
  material_id  TEXT NOT NULL REFERENCES resource(id),
  enabled     INTEGER NOT NULL DEFAULT 1,
  format      TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS lab_request (
  id          TEXT PRIMARY KEY,
  patient_id  TEXT NOT NULL REFERENCES patient(id),
  date        TEXT NOT NULL,
  pending     INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE IF NOT EXISTS request_exam (
  request_id TEXT NOT NULL REFERENCES lab_request(id) ON DELETE CASCADE,
  exam_id    TEXT NOT NULL REFERENCES exam(id),
  position   INTEGER NOT NULL,
```

CÓDIGO FUENTE

```
  PRIMARY KEY (request_id, exam_id)
);

CREATE TABLE IF NOT EXISTS sample (
  id          TEXT PRIMARY KEY,
  patient_id  TEXT NOT NULL REFERENCES patient(id),
  request_id  TEXT NOT NULL REFERENCES lab_request(id),
  exam_id     TEXT NOT NULL REFERENCES exam(id),
  taken_at    TEXT NOT NULL,
  state       TEXT NOT NULL,

  responsible TEXT NOT NULL,
  notes       TEXT,
  UNIQUE (request_id, exam_id)
);

CREATE TABLE IF NOT EXISTS sample_trace (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  sample_id TEXT NOT NULL REFERENCES sample(id) ON DELETE CASCADE,
  state     TEXT NOT NULL,
  at        TEXT NOT NULL,
  user_name TEXT NOT NULL,
  position  INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS sample_result (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  sample_id   TEXT NOT NULL REFERENCES sample(id) ON DELETE CASCADE,
  param_name  TEXT NOT NULL,
  value       TEXT,
  observation TEXT,
  UNIQUE (sample_id, param_name)
);

CREATE TABLE IF NOT EXISTS maintenance (
  id           TEXT PRIMARY KEY,
  equipment_id TEXT NOT NULL REFERENCES resource(id),
  type         TEXT NOT NULL,
  start_date   TEXT NOT NULL,
  end_date     TEXT NOT NULL,
  actual_date  TEXT,
  responsible  TEXT NOT NULL,
  provider     TEXT,
  description  TEXT,
  state        TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS audit_entry (
  id          TEXT PRIMARY KEY,
  at          TEXT NOT NULL,
  user_name   TEXT NOT NULL,
  role        TEXT NOT NULL,
  action      TEXT NOT NULL,
  module      TEXT NOT NULL,
  record_ref  TEXT NOT NULL,

  seq         INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS app_user (
  id     TEXT PRIMARY KEY,
  name   TEXT NOT NULL,
  email  TEXT NOT NULL,
  role   TEXT NOT NULL,
  active INTEGER NOT NULL DEFAULT 1
);

CREATE TABLE IF NOT EXISTS settings (
  id            INTEGER PRIMARY KEY CHECK (id = 1),
  lab           TEXT NOT NULL,
  branch        TEXT NOT NULL,
  notifications INTEGER NOT NULL DEFAULT 1
);

CREATE TABLE IF NOT EXISTS preference (
  name  TEXT PRIMARY KEY,
  value TEXT NOT NULL
```

CÓDIGO FUENTE

```
);

CREATE INDEX IF NOT EXISTS idx_sample_state   ON sample(state);
CREATE INDEX IF NOT EXISTS idx_sample_patient ON sample(patient_id);
CREATE INDEX IF NOT EXISTS idx_audit_at       ON audit_entry(at DESC);
CREATE INDEX IF NOT EXISTS idx_audit_seq      ON audit_entry(seq);
```

5. MODELO DE DATOS

Cada tabla tiene una entidad JPA. Se muestran las del flujo principal de atención.

domain/Puma.java

Ruta: src/main/java/pe/vizcacha/laboratorio/domain/Puma.java

Entidad Paciente.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

import java.time.LocalDate;
import java.time.Period;

@Entity
@Table(name = "patient")
public class Puma {

    @Id
    private String id;

    @Column(nullable = false, unique = true)
    private String dni;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @Column(name = "last_name", nullable = false)
    private String lastName;

    @Column(nullable = false)
    private LocalDate birth;

    @Column(nullable = false)
    private String sex;

    private String phone;
    private String email;
    private String address;
    private String notes;

    @Column(nullable = false)
    private LocalDate created;

    public int age(LocalDate reference) {
        return Period.between(birth, reference).getYears();
    }

    public String fullName() {
        return firstName + " " + lastName;
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public String getDni() { return dni; }
    public void setDni(String dni) { this.dni = dni; }
    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }
    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
    public LocalDate getBirth() { return birth; }
    public void setBirth(LocalDate birth) { this.birth = birth; }
    public String getSex() { return sex; }
    public void setSex(String sex) { this.sex = sex; }
    public String getPhone() { return phone; }
```

CÓDIGO FUENTE

```
    public void setPhone(String phone) { this.phone = phone; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }
    public String getNotes() { return notes; }
    public void setNotes(String notes) { this.notes = notes; }
    public LocalDate getCreated() { return created; }
    public void setCreated(LocalDate created) { this.created = created; }
}
```

domain/Condor.java

Ruta: src/main/java/pe/vizcacha/laboratorio/domain/Condor.java

Entidad Solicitud: paciente, fecha, exámenes solicitados (relación muchos a muchos) e indicador de pendiente de disponibilidad.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.JoinTable;
import jakarta.persistence.ManyToMany;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.OrderColumn;
import jakarta.persistence.Table;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "lab_request")
public class Condor {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "patient_id", nullable = false)
    private Puma patient;

    @Column(nullable = false)
    private LocalDate date;

    @Column(nullable = false)
    private boolean pending;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
            name = "request_exam",
            joinColumns = @JoinColumn(name = "request_id"),
            inverseJoinColumns = @JoinColumn(name = "exam_id"))
    @OrderColumn(name = "position")
    private List<Llama> exams = new ArrayList<>();

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public Puma getPatient() { return patient; }
    public void setPatient(Puma patient) { this.patient = patient; }
    public LocalDate getDate() { return date; }
    public void setDate(LocalDate date) { this.date = date; }
    public boolean isPending() { return pending; }
    public void setPending(boolean pending) { this.pending = pending; }
    public List<Llama> getExams() { return exams; }
    public void setExams(List<Llama> exams) { this.exams = exams; }
```

CÓDIGO FUENTE

```
}
```

domain/Vicuna.java

Ruta: src/main/java/pe/vizcacha/laboratorio/domain/Vicuna.java

Entidad Muestra: paciente, solicitud, examen, fecha de toma, estado, responsable, notas, pasos de trazabilidad y resultados.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.domain;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OrderBy;
import jakarta.persistence.Table;
import jakarta.persistence.UniqueConstraint;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "sample",
       uniqueConstraints = @UniqueConstraint(columnNames = {"request_id", "exam_id"}))
public class Vicuna {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "patient_id", nullable = false)
    private Puma patient;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "request_id", nullable = false)
    private Condor request;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "exam_id", nullable = false)
    private Llama exam;

    @Column(name = "taken_at", nullable = false)
    private LocalDateTime takenAt;

    @Column(nullable = false)
    private VicunaState state;

    @Column(nullable = false)
    private String responsible;

    private String notes;

    @OneToMany(mappedBy = "sample", cascade = CascadeType.ALL, orphanRemoval = true,
               fetch = FetchType.LAZY)
    @OrderBy("position ASC")
    private List<VicunaTrace> traces = new ArrayList<>();

    @OneToMany(mappedBy = "sample", cascade = CascadeType.ALL, orphanRemoval = true,
               fetch = FetchType.LAZY)
    @OrderBy("id ASC")
```

CÓDIGO FUENTE

```
    private List<VicunaResult> results = new ArrayList<>();

    public void addTrace(VicunaTrace trace) {
        trace.setSample(this);
        trace.setPosition(traces.size());
        traces.add(trace);
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public Puma getPatient() { return patient; }
    public void setPatient(Puma patient) { this.patient = patient; }
    public Condor getRequest() { return request; }
    public void setRequest(Condor request) { this.request = request; }
    public Llama getExam() { return exam; }
    public void setExam(Llama exam) { this.exam = exam; }
    public LocalDateTime getTakenAt() { return takenAt; }
    public void setTakenAt(LocalDateTime takenAt) { this.takenAt = takenAt; }
    public VicunaState getState() { return state; }
    public void setState(VicunaState state) { this.state = state; }
    public String getResponsible() { return responsible; }
    public void setResponsible(String responsible) { this.responsible = responsible; }
    public String getNotes() { return notes; }
    public void setNotes(String notes) { this.notes = notes; }
    public List<VicunaTrace> getTraces() { return traces; }
    public void setTraces(List<VicunaTrace> traces) { this.traces = traces; }
    public List<VicunaResult> getResults() { return results; }
    public void setResults(List<VicunaResult> results) { this.results = results; }
}
```

domain/VicunaTrace.java

Ruta: src/main/java/pe/vizcacha/laboratorio/domain/VicunaTrace.java

Paso de trazabilidad: estado alcanzado, fecha y hora y responsable.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;

import java.time.LocalDateTime;

@Entity
@Table(name = "sample_trace")
public class VicunaTrace {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "sample_id", nullable = false)
    private Vicuna sample;

    @Column(nullable = false)
    private VicunaState state;

    @Column(name = "at", nullable = false)
    private LocalDateTime at;

    @Column(name = "user_name", nullable = false)
```

CÓDIGO FUENTE

```
    private String userName;

    @Column(nullable = false)
    private int position;

    public VicunaTrace() {
    }

    public VicunaTrace(VicunaState state, LocalDateTime at, String userName) {
        this.state = state;
        this.at = at;
        this.userName = userName;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Vicuna getSample() { return sample; }
    public void setSample(Vicuna sample) { this.sample = sample; }
    public VicunaState getState() { return state; }
    public void setState(VicunaState state) { this.state = state; }
    public LocalDateTime getAt() { return at; }
    public void setAt(LocalDateTime at) { this.at = at; }
    public String getUserName() { return userName; }
    public void setUserName(String userName) { this.userName = userName; }
    public int getPosition() { return position; }
    public void setPosition(int position) { this.position = position; }
}
```

domain/VicunaResult.java

Ruta: src/main/java/pe/vizcacha/laboratorio/domain/VicunaResult.java

Valor de un parámetro del resultado con su observación.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import jakarta.persistence.UniqueConstraint;

@Entity
@Table(name = "sample_result",
       uniqueConstraints = @UniqueConstraint(columnNames = {"sample_id", "param_name"}))
public class VicunaResult {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "sample_id", nullable = false)
    private Vicuna sample;

    @Column(name = "param_name", nullable = false)
    private String paramName;

    private String value;

    private String observation;

    public VicunaResult() {
    }
```

CÓDIGO FUENTE

```
    public VicunaResult(String paramName, String value, String observation) {
        this.paramName = paramName;
        this.value = value;
        this.observation = observation;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Vicuna getSample() { return sample; }
    public void setSample(Vicuna sample) { this.sample = sample; }
    public String getParamName() { return paramName; }
    public void setParamName(String paramName) { this.paramName = paramName; }
    public String getValue() { return value; }
    public void setValue(String value) { this.value = value; }
    public String getObservation() { return observation; }
    public void setObservation(String observation) { this.observation = observation; }
}
```

domain/enums/VicunaState.java

Ruta: src/main/java/pe/vizcacha/laboratorio/domain/enums/VicunaState.java

Los cinco estados de una muestra en orden. next() devuelve el siguiente estado permitido.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.domain.enums;

public enum VicunaState implements LabeledEnum {

    RECIBIDA("RECIBIDA"),
    EN_ANALISIS("EN ANÁLISIS"),
    EN_VALIDACION("EN VALIDACIÓN"),
    VALIDADO("VALIDADO"),
    PUBLICADA("PUBLICADA");

    private final String label;

    VicunaState(String label) {
        this.label = label;
    }

    @Override
    public String label() {
        return label;
    }

    public VicunaState next() {
        return this == PUBLICADA ? null : values()[ordinal() + 1];
    }

    public static VicunaState fromLabel(String label) {
        for (VicunaState s : values()) {
            if (s.label.equals(label)) {
                return s;
            }
        }
        throw new IllegalArgumentException("Estado de muestra desconocido: " + label);
    }
}
```

repo/VicunaRepository.java

Ruta: src/main/java/pe/vizcacha/laboratorio/repo/VicunaRepository.java

Repositorio de muestras sobre Spring Data JPA.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.repo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;

import java.util.List;

public interface VicunaRepository extends JpaRepository<Vicuna, String> {

    List<Vicuna> findByState(VicunaState state);

    List<Vicuna> findByPatientId(String patientId);

    boolean existsByRequestIdAndExamId(String requestId, String examId);

    long countByState(VicunaState state);

    @Query("select s.id from Vicuna s")
    List<String> findAllIds();
}
```

6. REGLAS CENTRALES

service/AvailabilityService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/AvailabilityService.java

Regla central del sistema: un examen está disponible solo si está habilitado, su equipo está operativo, su reactivo y su material tienen existencias y el reactivo no está vencido. Cuando no lo está, devuelve los motivos que muestra la interfaz.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Llama;
import pe.vizcacha.laboratorio.domain.Alpaca;
import pe.vizcacha.laboratorio.domain.enums.EquipmentStatus;
import pe.vizcacha.laboratorio.domain.enums.AlpacaKind;
import pe.vizcacha.laboratorio.repo.LlamaRepository;

import java.util.ArrayList;
import java.util.List;

@Service
public class AvailabilityService {

    private final LlamaRepository exams;
    private final DemoClock clock;

    public AvailabilityService(LlamaRepository exams, DemoClock clock) {
        this.exams = exams;
        this.clock = clock;
    }

    public List<String> reasons(Llama exam) {
        List<String> motivos = new ArrayList<>();

        if (!exam.isEnabled()) {
            motivos.add("Examen deshabilitado");
        }

        Alpaca equipo = exam.getEquipment();
        if (equipo == null || equipo.getStatus() != EquipmentStatus.OPERATIVO) {
            String nombre = equipo == null ? "Equipo" : equipo.getName();
            boolean enMantenimiento = equipo != null
                    && equipo.getStatus() == EquipmentStatus.EN_MANTENIMIENTO;
            motivos.add(nombre + ": "
                    + (enMantenimiento ? "equipo en mantenimiento" : "equipo no
operativo"));
        }

        for (Alpaca r : List.of(exam.getReagent(), exam.getMaterial())) {
            motivos.addAll(reasonsFor(r));
        }

        return motivos;
    }

    private List<String> reasonsFor(Alpaca r) {
        if (r == null) {
            return List.of("Recurso no configurado");
        }
        if (r.getQty() < 1) {
            return List.of(r.getName() + ": " + (r.getKind() == AlpacaKind.REACTIVOS
                    ? "reactivo sin stock"
                    : "material insuficiente"));
        }
        if (r.getKind() == AlpacaKind.REACTIVOS
                && r.getExpires() != null
                && r.getExpires().isBefore(clock.today())) {
```

CÓDIGO FUENTE

```
            return List.of(r.getName() + ": producto vencido");
        }
        return List.of();
    }

    public boolean isAvailable(Llama exam) {
        return reasons(exam).isEmpty();
    }

    @Transactional(readOnly = true)
    public List<Llama> unavailableExams() {
        return exams.findAll().stream().filter(e -> !isAvailable(e)).toList();
    }
}
```

service/AlpacaStatusService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/AlpacaStatusService.java

Estado derivado de un recurso a partir de su cantidad, su mínimo y su vencimiento.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Alpaca;
import pe.vizcacha.laboratorio.domain.enums.AlpacaKind;

@Service
public class AlpacaStatusService {

    public static final String DISPONIBLE = "Disponible";
    public static final String STOCK_BAJO = "Stock bajo";
    public static final String AGOTADO = "Agotado";
    public static final String VENCIDO = "Vencido";

    private final DemoClock clock;

    public AlpacaStatusService(DemoClock clock) {
        this.clock = clock;
    }

    public String derived(Alpaca r) {
        if (r.getKind() == AlpacaKind.EQUIPOS) {
            return r.getStatus() == null ? "" : r.getStatus().label();
        }
        if (r.getKind() == AlpacaKind.REACTIVOS
                && r.getExpires() != null
                && r.getExpires().isBefore(clock.today())) {
            return VENCIDO;
        }
        if (r.getQty() == 0) {
            return AGOTADO;
        }
        if (r.getQty() <= r.getMinQty()) {
            return STOCK_BAJO;
        }
        return DISPONIBLE;
    }
}
```

service/IdSequence.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/IdSequence.java

Generador de códigos correlativos.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;

import java.util.Collection;

@Service
public class IdSequence {

    public String next(String prefix, Collection<String> existingIds) {
        long max = 0;
        for (String id : existingIds) {
            max = Math.max(max, numericSuffix(id));
        }
        return prefix + String.format("%06d", max + 1);
    }

    private long numericSuffix(String id) {
        if (id == null || id.isBlank()) {
            return 0;
        }
        String last = id.substring(id.lastIndexOf('-') + 1);
        try {
            return Long.parseLong(last);
        } catch (NumberFormatException e) {
            return 0;
        }
    }
}
```

7. FLUJO DE ATENCIÓN

service/PumaService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/PumaService.java

Registro y edición de pacientes. Valida que el DNI tenga 8 dígitos y no esté repetido.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Puma;
import pe.vizcacha.laboratorio.dto.PumaForm;
import pe.vizcacha.laboratorio.repo.PumaRepository;

import java.time.LocalDate;
import java.util.regex.Pattern;

@Service
public class PumaService {

    private static final Pattern OCHO_DIGITOS = Pattern.compile("\\d{8}");

    private final PumaRepository patients;
    private final BuhoService audit;
    private final SessionService session;
    private final IdSequence ids;
    private final DemoClock clock;

    public PumaService(PumaRepository patients, BuhoService audit,
                       SessionService session, IdSequence ids, DemoClock clock) {
        this.patients = patients;
        this.audit = audit;
        this.session = session;
        this.ids = ids;
        this.clock = clock;
    }

    @Transactional
    public Puma save(PumaForm form) {
        String dni = trim(form.dni());
        if (!OCHO_DIGITOS.matcher(dni).matches()) {
            throw new BusinessException("El DNI debe tener exactamente 8 dígitos.");
        }

        boolean nuevo = isBlank(form.id());

        patients.findByDni(dni)
                .filter(otro -> nuevo || !otro.getId().equals(form.id()))
                .ifPresent(otro -> {
                    throw new BusinessException(
                            "Ya existe un paciente con este DNI. Puedes consultarlo en
Pacientes.");
                });

        Puma p = nuevo
                ? new Puma()
                : patients.findById(form.id()).orElseThrow(() ->
                        new BusinessException("El paciente indicado no existe."));

        if (nuevo) {
            p.setId(ids.next("PAC-", patients.findAllIds()));

            p.setCreated(clock.today());
        }

        p.setDni(dni);
        p.setFirstName(trim(form.first()));
        p.setLastName(trim(form.last()));
        p.setBirth(LocalDate.parse(trim(form.birth())));
        p.setSex(trim(form.sex()));
```

CÓDIGO FUENTE

```
        p.setPhone(trim(form.phone()));
        p.setEmail(trim(form.email()));
        p.setAddress(trim(form.address()));
        p.setNotes(trim(form.notes()));

        Puma guardado = patients.save(p);

        audit.record(session.userName(), session.role(),
                nuevo ? "Paciente creado" : "Paciente actualizado",
                "Pacientes", guardado.getId());

        return guardado;
    }

    private String trim(String s) {
        return s == null ? "" : s.trim();
    }

    private boolean isBlank(String s) {
        return s == null || s.isBlank();
    }
}
```

service/CondorService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/CondorService.java

Creación de solicitudes: valida paciente y exámenes, comprueba la disponibilidad y permite registrar como pendiente.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Llama;
import pe.vizcacha.laboratorio.domain.Condor;
import pe.vizcacha.laboratorio.domain.Puma;
import pe.vizcacha.laboratorio.dto.NewCondorForm;
import pe.vizcacha.laboratorio.repo.LlamaRepository;
import pe.vizcacha.laboratorio.repo.CondorRepository;
import pe.vizcacha.laboratorio.repo.PumaRepository;

import java.util.ArrayList;
import java.util.List;

@Service
public class CondorService {

    private static final String FALTA_SELECCION = "Selecciona un paciente y al menos un
examen.";

    private final CondorRepository requests;
    private final PumaRepository patients;
    private final LlamaRepository exams;
    private final AvailabilityService availability;
    private final BuhoService audit;
    private final SessionService session;
    private final IdSequence ids;
    private final DemoClock clock;

    public CondorService(CondorRepository requests, PumaRepository patients,
                         LlamaRepository exams, AvailabilityService availability,
                         BuhoService audit, SessionService session,
                         IdSequence ids, DemoClock clock) {
        this.requests = requests;
        this.patients = patients;
        this.exams = exams;
```

CÓDIGO FUENTE

```
        this.availability = availability;
        this.audit = audit;
        this.session = session;
        this.ids = ids;
        this.clock = clock;
    }

    @Transactional
    public Condor create(NewCondorForm form) {
        if (form.patientId() == null || form.patientId().isBlank()
                || form.examIds() == null || form.examIds().isEmpty()) {
            throw new BusinessException(FALTA_SELECCION);
        }

        Puma paciente = patients.findById(form.patientId())
                .orElseThrow(() -> new BusinessException(FALTA_SELECCION));

        List<Llama> elegidos = new ArrayList<>();
        for (String examId : form.examIds()) {
            elegidos.add(exams.findById(examId)
                    .orElseThrow(() -> new BusinessException(FALTA_SELECCION)));
        }

        boolean pendiente = elegidos.stream().anyMatch(e -> !availability.isAvailable(e));

        if (pendiente && !form.allowPending()) {
            throw new BusinessException(
                    "Confirma el registro pendiente de disponibilidad para continuar.");
        }

        Condor r = new Condor();
        r.setId(ids.next("SOL-", requests.findAllIds()));
        r.setPatient(paciente);
        r.setDate(clock.today());
        r.setPending(pendiente);
        r.setExams(elegidos);

        Condor guardada = requests.save(r);

        audit.record(session.userName(), session.role(),
                "Solicitud creada", "Solicitudes", guardada.getId());

        return guardada;
    }
}
```

service/VicunaService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/VicunaService.java

Registro de muestras (con descuento de reactivo y material) y cambios de estado, que solo admiten el paso siguiente del flujo.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Llama;
import pe.vizcacha.laboratorio.domain.Condor;
import pe.vizcacha.laboratorio.domain.Alpaca;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.domain.VicunaTrace;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;
import pe.vizcacha.laboratorio.dto.NewVicunaForm;
import pe.vizcacha.laboratorio.repo.LlamaRepository;
import pe.vizcacha.laboratorio.repo.CondorRepository;
import pe.vizcacha.laboratorio.repo.VicunaRepository;
```

CÓDIGO FUENTE

```
import java.time.LocalDateTime;
import java.util.List;

@Service
public class VicunaService {

    private static final String FALTA_SELECCION = "Selecciona una solicitud y un examen.";

    private final VicunaRepository samples;
    private final CondorRepository requests;
    private final LlamaRepository exams;
    private final AvailabilityService availability;
    private final BuhoService audit;
    private final SessionService session;
    private final IdSequence ids;
    private final DemoClock clock;

    public VicunaService(VicunaRepository samples, CondorRepository requests,
                         LlamaRepository exams, AvailabilityService availability,
                         BuhoService audit,
                         SessionService session, IdSequence ids, DemoClock clock) {
        this.samples = samples;
        this.requests = requests;
        this.exams = exams;
        this.availability = availability;
        this.audit = audit;
        this.session = session;
        this.ids = ids;
        this.clock = clock;
    }

    @Transactional
    public Vicuna register(NewVicunaForm form) {
        if (form.requestId() == null || form.requestId().isBlank()
                || form.examId() == null || form.examId().isBlank()) {
            throw new BusinessException(FALTA_SELECCION);
        }

        Condor solicitud = requests.findById(form.requestId())
                .orElseThrow(() -> new BusinessException(FALTA_SELECCION));
        Llama examen = exams.findById(form.examId())
                .orElseThrow(() -> new BusinessException(FALTA_SELECCION));

        List<String> motivos = availability.reasons(examen);
        if (!motivos.isEmpty()) {
            throw new BusinessException("No se puede registrar la toma: "
                    + String.join("; ", motivos) + ". La solicitud permanece pendiente.");
        }

        if (samples.existsByRequestIdAndExamId(solicitud.getId(), examen.getId())) {
            throw new BusinessException(
                    "Este examen ya tiene una muestra registrada en la solicitud.");
        }

        Vicuna s = new Vicuna();

        s.setId(ids.next("MUE-" + clock.today().getYear() + "-", samples.findAllIds()));
        s.setPatient(solicitud.getPatient());
        s.setRequest(solicitud);
        s.setExam(examen);
        s.setTakenAt(parseTakenAt(form.takenAt()));
        s.setState(VicunaState.RECIBIDA);
        s.setResponsible(blankTo(form.responsible(), session.userName()));
        s.setNotes("");

        s.addTrace(new VicunaTrace(VicunaState.RECIBIDA, clock.timestamp(),
session.userName()));

        examen.getReagent().setQty(examen.getReagent().getQty() - 1);
        examen.getMaterial().setQty(examen.getMaterial().getQty() - 1);

        Vicuna guardada = samples.save(s);
```

CÓDIGO FUENTE

```
        recalculatePending(solicitud);

        audit.record(session.userName(), session.role(),
                "Muestra registrada", "Muestras", guardada.getId());

        return guardada;
    }

    @Transactional
    public Vicuna transition(String sampleId, String stateLabel) {
        Vicuna s = samples.findById(sampleId)
                .orElseThrow(() -> new BusinessException("La muestra indicada no
existe."));

        VicunaState destino = VicunaState.fromLabel(stateLabel);
        VicunaState siguiente = s.getState().next();

        if (destino != siguiente) {
            throw new BusinessException(siguiente == null
                    ? "La muestra ya está publicada: no admite más transiciones."
                    : "Transición no válida: una muestra en " + s.getState().label()
                            + " solo puede pasar a " + siguiente.label() + ".");
        }

        if (destino == VicunaState.EN_ANALISIS) {
            List<String> motivos = availability.reasons(s.getExam());
            if (!motivos.isEmpty()) {
                throw new BusinessException(
                        "No se puede iniciar el análisis: " + String.join("; ", motivos));
            }
        }

        s.setState(destino);
        s.addTrace(new VicunaTrace(destino, clock.timestamp(), session.userName()));

        audit.record(session.userName(), session.role(),
                actionFor(destino),
                destino == VicunaState.EN_ANALISIS ? "Muestras" : "Resultados",
                s.getId());

        return s;
    }

    private String actionFor(VicunaState destino) {
        return switch (destino) {
            case PUBLICADA -> "Resultado publicado";
            case VALIDADO -> "Resultado validado";
            default -> "Muestra enviada a análisis";
        };
    }

    void recalculatePending(Condor solicitud) {
        solicitud.setPending(solicitud.getExams().stream()
                .anyMatch(e -> !availability.isAvailable(e)));
    }

    private LocalDateTime parseTakenAt(String raw) {
        if (raw == null || raw.isBlank()) {
            return clock.timestamp();
        }
        String texto = raw.trim();
        return LocalDateTime.parse(texto.length() == 16 ? texto + ":00" : texto);
    }

    private String blankTo(String value, String fallback) {
        return value == null || value.isBlank() ? fallback : value.trim();
    }
}
```

service/ResultService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/ResultService.java

Guardado de resultados: actualiza los valores existentes sin duplicarlos, guarda como borrador o envía a validación.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.domain.VicunaResult;
import pe.vizcacha.laboratorio.domain.VicunaTrace;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;
import pe.vizcacha.laboratorio.dto.ResultForm;
import pe.vizcacha.laboratorio.repo.VicunaRepository;

import java.util.List;
import java.util.Map;

@Service
public class ResultService {

    private static final String SUFIJO_OBSERVACION = "_obs";

    private static final String MUESTRA_NO_VALIDA =
            "Selecciona una muestra en análisis para ingresar resultados.";

    private final VicunaRepository samples;
    private final ResultFormatCatalog formats;
    private final BuhoService audit;
    private final SessionService session;
    private final DemoClock clock;

    public ResultService(VicunaRepository samples, ResultFormatCatalog formats,
                         BuhoService audit,
                         SessionService session, DemoClock clock) {
        this.samples = samples;
        this.formats = formats;
        this.audit = audit;
        this.session = session;
        this.clock = clock;
    }

    @Transactional
    public Vicuna save(String sampleId, ResultForm form) {
        Vicuna s = samples.findById(sampleId)
                .orElseThrow(() -> new BusinessException(MUESTRA_NO_VALIDA));

        if (s.getState() != VicunaState.EN_ANALISIS) {
            throw new BusinessException(MUESTRA_NO_VALIDA);
        }

        Map<String, String> valores = form.values() == null ? Map.of() : form.values();
        List<LlamaParam> params = formats.params(s.getExam().getFormat());

        if (form.submit() && params.stream().anyMatch(p -> isBlank(valores.get(p.name())))
{
            throw new BusinessException(
                    "Completa todos los parámetros antes de enviar a validación.");
        }

        boolean numeroInvalido = params.stream().anyMatch(p ->
                p.isNumeric() && !isBlank(valores.get(p.name()))
                        && !isValidNonNegative(valores.get(p.name())));
        if (numeroInvalido) {
            throw new BusinessException(
                    "Los parámetros numéricos deben contener valores válidos, no
negativos.");
        }

        java.util.Set<String> vigentes = new java.util.HashSet<>();
        for (LlamaParam p : params) {
            vigentes.add(p.name());
        }
        s.getResults().removeIf(r -> !vigentes.contains(r.getParamName()));
```

CÓDIGO FUENTE

```
        for (LlamaParam p : params) {
            String valor = valores.get(p.name());
            String observacion = valores.get(p.name() + SUFIJO_OBSERVACION);
            VicunaResult r = s.getResults().stream()
                    .filter(x -> x.getParamName().equals(p.name()))
                    .findFirst()
                    .orElse(null);
            if (r == null) {
                r = new VicunaResult();
                r.setParamName(p.name());
                r.setSample(s);
                s.getResults().add(r);
            }
            r.setValue(valor == null ? "" : valor.trim());
            r.setObservation(isBlank(observacion) ? null : observacion.trim());
        }

        s.setNotes(form.notes() == null ? "" : form.notes());

        if (form.submit()) {
            s.setState(VicunaState.EN_VALIDACION);
            s.addTrace(new VicunaTrace(VicunaState.EN_VALIDACION,
                    clock.timestamp(), session.userName()));
        }

        audit.record(session.userName(), session.role(),
                form.submit() ? "Resultado ingresado" : "Borrador de resultado guardado",
                "Resultados", s.getId());

        return s;
    }

    private boolean isBlank(String s) {
        return s == null || s.trim().isEmpty();
    }

    private boolean isValidNonNegative(String raw) {
        try {
            double v = Double.parseDouble(raw.trim());
            return Double.isFinite(v) && v >= 0;
        } catch (NumberFormatException e) {
            return false;
        }
    }
}
```

8. INFORME DE RESULTADOS Y EXPORTACIONES

service/ResultReportService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/ResultReportService.java

Arma el informe de una muestra publicada: datos del paciente y de la muestra, parámetros con unidad e intervalo, marca ALTO o BAJO y las tres firmas tomadas de la trazabilidad. Rechaza las muestras no publicadas.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.domain.VicunaResult;
import pe.vizcacha.laboratorio.domain.VicunaTrace;
import pe.vizcacha.laboratorio.domain.Settings;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;
import pe.vizcacha.laboratorio.dto.ResultReportDto;
import pe.vizcacha.laboratorio.repo.VicunaRepository;
import pe.vizcacha.laboratorio.repo.SettingsRepository;

import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@Service
public class ResultReportService {

    private static final String AVISO =
            "Este documento no constituye un informe clínico ni un diagnóstico médico. "
                    + "Los valores fuera del intervalo de referencia solo señalan una
desviación.";

    private static final DateTimeFormatter FECHA_HORA =
            DateTimeFormatter.ofPattern("dd/MM/yyyy · HH:mm");

    private static final DateTimeFormatter SOLO_FECHA =
            DateTimeFormatter.ofPattern("dd/MM/yyyy");

    private final VicunaRepository samples;
    private final SettingsRepository settings;
    private final ResultFormatCatalog formats;
    private final DemoClock clock;
    private final SessionService session;

    public ResultReportService(VicunaRepository samples, SettingsRepository settings,
                               ResultFormatCatalog formats, DemoClock clock,
                               SessionService session) {
        this.session = session;
        this.samples = samples;
        this.settings = settings;
        this.formats = formats;
        this.clock = clock;
    }

    @Transactional(readOnly = true)
    public ResultReportDto build(String sampleId) {
        Vicuna s = samples.findById(sampleId)
                .orElseThrow(() -> new BusinessException("La muestra indicada no
existe."));

        if (s.getState() != VicunaState.PUBLICADA) {
            throw new BusinessException(
                    "Solo se puede imprimir un resultado publicado.");
        }
```

CÓDIGO FUENTE

```
        Settings config = settings.findById(Settings.SINGLETON_ID).orElse(null);

        Map<String, VicunaResult> porParametro = new LinkedHashMap<>();
        for (VicunaResult r : s.getResults()) {
            porParametro.put(r.getParamName(), r);
        }

        List<ResultReportDto.Row> filas = new ArrayList<>();
        List<String> fuera = new ArrayList<>();

        for (LlamaParam p : formats.params(s.getExam().getFormat())) {
            VicunaResult r = porParametro.get(p.name());
            String valor = r == null || r.getValue() == null ? "" : r.getValue();
            String marca = flagFor(p, valor);
            boolean desviado = !marca.isEmpty();
            if (desviado) {
                fuera.add(p.name());
            }
            filas.add(new ResultReportDto.Row(
                    p.name(),
                    valor.isBlank() ? "—" : valor,
                    p.unit(),
                    p.reference(),
                    r == null || r.getObservation() == null || r.getObservation().isBlank()
                            ? "—" : r.getObservation(),
                    desviado,
                    marca));
        }

        return new ResultReportDto(
                config == null ? SettingsService.LAB_POR_DEFECTO : config.getLab(),
                config == null ? "Laboratorio Central" : config.getBranch(),
                s.getId(),
                clock.timestamp().format(FECHA_HORA),
                s.getPatient().fullName(),
                s.getPatient().getDni(),
                s.getPatient().getBirth().format(SOLO_FECHA),
                s.getPatient().age(clock.today()),
                s.getPatient().getSex(),
                s.getId(),
                s.getExam().getName(),
                s.getExam().getCategory(),
                s.getExam().getSampleType(),
                s.getTakenAt().format(FECHA_HORA),
                firmaFecha(s, VicunaState.PUBLICADA),
                firmaUsuario(s, VicunaState.EN_VALIDACION),
                firmaUsuario(s, VicunaState.VALIDADO),
                firmaUsuario(s, VicunaState.PUBLICADA),
                filas,
                s.getNotes() == null || s.getNotes().isBlank() ? "—" : s.getNotes(),
                fuera,
                AVISO);
    }

    private String flagFor(LlamaParam p, String valor) {
        if (!p.isNumeric() || valor == null || valor.isBlank()) {
            return "";
        }
        try {
            double v = Double.parseDouble(valor.trim());
            if (v > p.max()) {
                return "ALTO";
            }
            if (v < p.min()) {
                return "BAJO";
            }
            return "";
        } catch (NumberFormatException e) {
            return "";
        }
    }

    private String firmaUsuario(Vicuna s, VicunaState estado) {
        return traza(s, estado).map(VicunaTrace::getUserName)
                .map(n -> SessionService.REGISTRO_LOCAL.equals(n) ? session.userName() : n)
                .orElse("—");
    }

    private String firmaFecha(Vicuna s, VicunaState estado) {
```

CÓDIGO FUENTE

```
        return traza(s, estado).map(t -> t.getAt().format(FECHA_HORA)).orElse("—");
    }

    private java.util.Optional<VicunaTrace> traza(Vicuna s, VicunaState estado) {
        return s.getTraces().stream().filter(t -> t.getState() == estado).findFirst();
    }
}
```

service/ExportService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/ExportService.java

Genera los archivos: el PDF del informe de resultados, y los reportes en PDF (OpenPDF) y Excel (Apache POI).

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import com.lowagie.text.Chunk;
import com.lowagie.text.Document;
import com.lowagie.text.Element;
import com.lowagie.text.Font;
import com.lowagie.text.FontFactory;
import com.lowagie.text.PageSize;
import com.lowagie.text.Paragraph;
import com.lowagie.text.Phrase;
import com.lowagie.text.Rectangle;
import com.lowagie.text.pdf.PdfPCell;
import com.lowagie.text.pdf.PdfPTable;
import com.lowagie.text.pdf.PdfPageEventHelper;
import com.lowagie.text.pdf.PdfWriter;
import org.apache.poi.ss.usermodel.BorderStyle;
import org.apache.poi.ss.usermodel.CellStyle;
import org.apache.poi.ss.usermodel.FillPatternType;
import org.apache.poi.ss.usermodel.HorizontalAlignment;
import org.apache.poi.ss.usermodel.IndexedColors;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.dto.ExportResultDto;
import pe.vizcacha.laboratorio.dto.ReportDto;
import pe.vizcacha.laboratorio.dto.ResultReportDto;

import java.awt.Color;
import java.io.IOException;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.text.Normalizer;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.Locale;

@Service
public class ExportService {

    public static final String PDF = "pdf";
    public static final String XLSX = "xlsx";

    private static final Color FONDO_CABECERA = new Color(0x27, 0x33, 0x35);
    private static final Color TURQUESA = new Color(0x6f, 0x9e, 0x93);
    private static final Color GRIS_SUAVE = new Color(0xf2, 0xf6, 0xf5);

    private static final Color TURQUESA_OSCURO = new Color(0x35, 0x68, 0x5f);
    private static final Color AMBAR = new Color(0x8a, 0x61, 0x00);
```

CÓDIGO FUENTE

```
    private static final DateTimeFormatter FECHA_ES_PE =
            DateTimeFormatter.ofPattern("dd/MM/yyyy");
    private static final DateTimeFormatter SELLO_ARCHIVO =
            DateTimeFormatter.ofPattern("yyyyMMdd-HHmmss");


    private final DemoClock clock;
    private final PreferenceService preferences;
    private final String exportDirPorDefecto;


    public ExportService(DemoClock clock, PreferenceService preferences,
                         @Value("${app.export-dir:exportaciones}") String exportDir) {
        this.clock = clock;
        this.preferences = preferences;
        this.exportDirPorDefecto = exportDir;
    }


    public Path carpeta() {
        return Paths.get(preferences.get(
                pe.vizcacha.laboratorio.domain.Preference.CARPETA_EXPORTACIONES,
                exportDirPorDefecto));
    }


    public ExportResultDto export(ReportDto report, String format, String lab, String
branch) {
        String formato = formato(format);
        return exportTo(report, formato, lab, branch, carpeta().resolve(fileName(report,
formato)));
    }


    public String formato(String format) {
        String formato = format == null ? PDF : format.trim().toLowerCase(Locale.ROOT);
        if (!PDF.equals(formato) && !XLSX.equals(formato)) {
            throw new BusinessException("Formato de exportación no soportado: " + format);
        }
        return formato;
    }


    public String reportFileName(ReportDto report, String format) {
        return fileName(report, formato(format));
    }


    public ExportResultDto exportTo(ReportDto report, String format, String lab, String
branch,
                                    Path archivo) {
        String formato = formato(format);
        Path carpeta = archivo.toAbsolutePath().getParent();
        try {
            Files.createDirectories(carpeta);
            try (OutputStream out = Files.newOutputStream(archivo)) {
                if (PDF.equals(formato)) {
                    writePdf(report, lab, branch, out);
                } else {
                    writeXlsx(report, lab, branch, out);
                }
            }
            return new ExportResultDto(archivo.getFileName().toString(),
                    archivo.toAbsolutePath().toString(), Files.size(archivo));
        } catch (IOException e) {
            throw new BusinessException("No se pudo escribir el archivo: " +
e.getMessage());
        }
    }


    public ExportResultDto exportResult(ResultReportDto informe) {
        return exportResultTo(informe, carpeta().resolve(resultFileName(informe)));
    }


    public String resultFileName(ResultReportDto informe) {
        return "Resultado-" + informe.reportNumber() + ".pdf";
    }


    public byte[] resultPdf(ResultReportDto informe) {
        java.io.ByteArrayOutputStream out = new java.io.ByteArrayOutputStream();
        writeResultPdf(informe, out);
        return out.toByteArray();
    }
```

CÓDIGO FUENTE

```
    public ExportResultDto exportResultTo(ResultReportDto informe, Path archivo) {
        Path carpeta = archivo.toAbsolutePath().getParent();

        try {
            Files.createDirectories(carpeta);
            try (OutputStream out = Files.newOutputStream(archivo)) {
                writeResultPdf(informe, out);
            }
            return new ExportResultDto(archivo.getFileName().toString(),
                    archivo.toAbsolutePath().toString(), Files.size(archivo));
        } catch (IOException e) {
            throw new BusinessException("No se pudo escribir el informe: " +
e.getMessage());
        }
    }

    private void writeResultPdf(ResultReportDto r, OutputStream out) {
        Document doc = new Document(PageSize.A4, 42, 42, 40, 54);
        PdfWriter writer = PdfWriter.getInstance(doc, out);
        writer.setPageEvent(new PieDeInforme(r));
        doc.addTitle("Informe de laboratorio · " + r.reportNumber());
        doc.addCreator(r.lab());
        doc.open();

        doc.add(encabezado(r));
        doc.add(tituloInforme());
        doc.add(datosEnDosPaneles(r));
        doc.add(bandaDeExamen(r));
        doc.add(tablaDeResultados(r));
        doc.add(leyendaDeMarcas(r));
        doc.add(observaciones(r));
        doc.add(firmas(r));

        doc.close();
    }

    private Font f(String familia, float tam, Color color) {
        return FontFactory.getFont(familia, tam, color);
    }

    private PdfPTable encabezado(ResultReportDto r) {
        PdfPTable tabla = new PdfPTable(new float[] {6f, 4f});
        tabla.setWidthPercentage(100);

        PdfPCell marca = new PdfPCell();
        marca.setBorder(Rectangle.NO_BORDER);
        marca.addElement(new Paragraph(r.lab().toUpperCase(Locale.ROOT),
                f(FontFactory.HELVETICA_BOLD, 17, FONDO_CABECERA)));
        Paragraph sede = new Paragraph(r.branch(),
                f(FontFactory.HELVETICA, 8, Color.DARK_GRAY));
        sede.setSpacingBefore(2);
        marca.addElement(sede);
        tabla.addCell(marca);

        PdfPCell numero = new PdfPCell();
        numero.setBorder(Rectangle.NO_BORDER);
        numero.setHorizontalAlignment(Element.ALIGN_RIGHT);
        Paragraph etiqueta = new Paragraph("INFORME N.º",
                f(FontFactory.HELVETICA, 7, Color.GRAY));
        etiqueta.setAlignment(Element.ALIGN_RIGHT);
        numero.addElement(etiqueta);
        Paragraph valor = new Paragraph(r.reportNumber(),
                f(FontFactory.COURIER_BOLD, 12, FONDO_CABECERA));
        valor.setAlignment(Element.ALIGN_RIGHT);
        numero.addElement(valor);
        tabla.addCell(numero);

        PdfPCell regla = new PdfPCell();
        regla.setColspan(2);
        regla.setFixedHeight(2.4f);
        regla.setBackgroundColor(TURQUESA);
        regla.setBorder(Rectangle.NO_BORDER);
        regla.setPaddingTop(8);
        tabla.addCell(regla);
```

CÓDIGO FUENTE

```
        return tabla;
    }

    private Paragraph tituloInforme() {
        Paragraph titulo = new Paragraph("INFORME DE RESULTADOS DE LABORATORIO",
                f(FontFactory.HELVETICA_BOLD, 11, FONDO_CABECERA));
        titulo.setAlignment(Element.ALIGN_CENTER);
        titulo.setSpacingBefore(16);
        titulo.setSpacingAfter(14);
        return titulo;
    }

    private PdfPTable datosEnDosPaneles(ResultReportDto r) {
        PdfPTable contenedor = new PdfPTable(2);
        contenedor.setWidthPercentage(100);
        contenedor.setSpacingAfter(16);

        contenedor.addCell(panel("DATOS DEL PACIENTE", new String[][] {
                {"Nombre", r.patientName()},
                {"Documento", r.patientDni()},
                {"Fecha de nacimiento", r.patientBirth()},
                {"Edad / Sexo", r.patientAge() + " años · " + r.patientSex()},
        }));
        contenedor.addCell(panel("DATOS DE LA MUESTRA", new String[][] {
                {"Código", r.sampleId()},
                {"Tipo de muestra", r.sampleType()},
                {"Fecha de toma", r.takenAt()},
                {"Fecha de emisión", r.issuedAt()},
        }));
        return contenedor;
    }

    private PdfPCell panel(String titulo, String[][] pares) {
        PdfPCell celda = new PdfPCell();
        celda.setBorderWidth(0.6f);
        celda.setBorderColor(new Color(0xc8, 0xd4, 0xd3));
        celda.setPadding(10);

        Paragraph cabecera = new Paragraph(titulo,
                f(FontFactory.HELVETICA_BOLD, 7, TURQUESA_OSCURO));
        cabecera.setSpacingAfter(7);
        celda.addElement(cabecera);

        PdfPTable datos = new PdfPTable(new float[] {4.2f, 6f});
        datos.setWidthPercentage(100);
        for (String[] par : pares) {
            datos.addCell(celdaDato(par[0] + ":", f(FontFactory.HELVETICA, 8,
Color.GRAY)));
            datos.addCell(celdaDato(par[1], f(FontFactory.HELVETICA_BOLD, 8.5f,
Color.BLACK)));
        }
        celda.addElement(datos);
        return celda;
    }

    private PdfPCell celdaDato(String texto, Font fuente) {
        PdfPCell c = new PdfPCell(new Phrase(texto, fuente));
        c.setBorder(Rectangle.NO_BORDER);
        c.setPaddingBottom(3.5f);
        c.setPaddingTop(0);
        return c;
    }

    private PdfPTable bandaDeExamen(ResultReportDto r) {
        PdfPTable banda = new PdfPTable(1);
        banda.setWidthPercentage(100);
        banda.setSpacingAfter(0);

        PdfPCell celda = new PdfPCell(new Phrase(
                r.examCategory().toUpperCase(Locale.ROOT) + "  ·  "
                        + r.examName().toUpperCase(Locale.ROOT),
                f(FontFactory.HELVETICA_BOLD, 8.5f, Color.WHITE)));
        celda.setBackgroundColor(FONDO_CABECERA);
        celda.setPadding(8);
        celda.setBorder(Rectangle.NO_BORDER);
        banda.addCell(celda);
        return banda;
    }
```

CÓDIGO FUENTE

```
    private PdfPTable tablaDeResultados(ResultReportDto r) {
        PdfPTable tabla = new PdfPTable(new float[] {3f, 1.9f, 1.2f, 2.2f, 3f});
        tabla.setWidthPercentage(100);

        Font cabecera = f(FontFactory.HELVETICA_BOLD, 7, Color.GRAY);
        for (String c : new String[] {"PARÁMETRO", "RESULTADO", "UNIDAD",
                "VALORES DE REFERENCIA", "OBSERVACIÓN"}) {
            PdfPCell celda = new PdfPCell(new Phrase(c, cabecera));
            celda.setBorder(Rectangle.BOTTOM);
            celda.setBorderWidthBottom(1.1f);
            celda.setBorderColorBottom(FONDO_CABECERA);
            celda.setPadding(7);
            celda.setPaddingTop(10);
            tabla.addCell(celda);
        }
        tabla.setHeaderRows(1);

        Font normal = f(FontFactory.HELVETICA, 9, Color.BLACK);
        Font tenue = f(FontFactory.HELVETICA, 8.5f, new Color(0x4a, 0x58, 0x57));
        Font valorNormal = f(FontFactory.HELVETICA_BOLD, 9.5f, Color.BLACK);
        Font valorDesviado = f(FontFactory.HELVETICA_BOLD, 9.5f, AMBAR);
        Font marcaDesviada = f(FontFactory.HELVETICA_BOLD, 7, AMBAR);

        for (ResultReportDto.Row fila : r.rows()) {
            Phrase resultado = new Phrase();
            resultado.add(new Chunk(fila.value(),
                    fila.outOfRange() ? valorDesviado : valorNormal));
            if (!fila.flag().isEmpty()) {
                resultado.add(new Chunk("   " + fila.flag(), marcaDesviada));
            }

            tabla.addCell(celdaResultado(new Phrase(fila.name(), normal)));
            tabla.addCell(celdaResultado(resultado));
            tabla.addCell(celdaResultado(new Phrase(fila.unit(), tenue)));
            tabla.addCell(celdaResultado(new Phrase(fila.reference(), tenue)));
            tabla.addCell(celdaResultado(new Phrase(fila.observation(), tenue)));
        }
        return tabla;
    }

    private PdfPCell celdaResultado(Phrase contenido) {
        PdfPCell celda = new PdfPCell(contenido);
        celda.setBorder(Rectangle.BOTTOM);
        celda.setBorderWidthBottom(0.4f);
        celda.setBorderColorBottom(new Color(0xdd, 0xe5, 0xe4));
        celda.setPadding(7);
        return celda;
    }

    private Paragraph leyendaDeMarcas(ResultReportDto r) {
        Paragraph leyenda = new Paragraph(r.outOfRange().isEmpty()
                ? "Todos los parámetros se encuentran dentro del intervalo de referencia."
                : "ALTO / BAJO: valor fuera del intervalo de referencia configurado. "
                        + "Este indicador señala una desviación y no constituye un "
                        + "diagnóstico médico.",
                f(FontFactory.HELVETICA_OBLIQUE, 7.5f,
                        r.outOfRange().isEmpty() ? Color.GRAY : AMBAR));
        leyenda.setSpacingBefore(8);
        return leyenda;
    }

    private PdfPTable observaciones(ResultReportDto r) {
        PdfPTable tabla = new PdfPTable(1);
        tabla.setWidthPercentage(100);
        tabla.setSpacingBefore(16);

        PdfPCell titulo = new PdfPCell(new Phrase("OBSERVACIONES GENERALES",
                f(FontFactory.HELVETICA_BOLD, 7, TURQUESA_OSCURO)));
        titulo.setBorder(Rectangle.NO_BORDER);
        titulo.setPaddingBottom(4);
        tabla.addCell(titulo);

        PdfPCell texto = new PdfPCell(new Phrase(r.notes(),
                f(FontFactory.HELVETICA, 9, Color.BLACK)));
        texto.setBorder(Rectangle.TOP);
        texto.setBorderWidthTop(0.6f);
```

CÓDIGO FUENTE

```
        texto.setBorderColorTop(new Color(0xc8, 0xd4, 0xd3));
        texto.setPadding(8);
        texto.setMinimumHeight(30);
        tabla.addCell(texto);
        return tabla;
    }

    private PdfPTable firmas(ResultReportDto r) {
        PdfPTable tabla = new PdfPTable(3);
        tabla.setWidthPercentage(100);
        tabla.setSpacingBefore(34);

        String[][] firmas = {
                {r.enteredBy(), "Ingreso de resultados"},
                {r.validatedBy(), "Validación técnica"},
                {r.publishedBy(), "Publicación · " + r.publishedAt()},
        };
        for (String[] firma : firmas) {
            PdfPCell celda = new PdfPCell();
            celda.setBorder(Rectangle.TOP);
            celda.setBorderWidthTop(0.6f);
            celda.setBorderColorTop(new Color(0x9a, 0xaa, 0xa9));
            celda.setPaddingTop(6);
            celda.setPaddingLeft(10);
            celda.setPaddingRight(10);

            Paragraph nombre = new Paragraph(firma[0],
                    f(FontFactory.HELVETICA_BOLD, 8.5f, Color.BLACK));
            nombre.setAlignment(Element.ALIGN_CENTER);
            celda.addElement(nombre);

            Paragraph rol = new Paragraph(firma[1],
                    f(FontFactory.HELVETICA, 7, Color.GRAY));
            rol.setAlignment(Element.ALIGN_CENTER);
            celda.addElement(rol);

            tabla.addCell(celda);
        }
        return tabla;
    }

    private final class PieDeInforme extends PdfPageEventHelper {

        private final ResultReportDto informe;

        private PieDeInforme(ResultReportDto informe) {
            this.informe = informe;
        }

        @Override
        public void onEndPage(PdfWriter writer, Document document) {
            Font legal = FontFactory.getFont(FontFactory.HELVETICA, 6.5f, Color.GRAY);
            PdfPTable pie = new PdfPTable(new float[] {8f, 2f});
            pie.setTotalWidth(document.right() - document.left());

            PdfPCell aviso = new PdfPCell(new Phrase(informe.disclaimer(), legal));
            aviso.setBorder(Rectangle.TOP);
            aviso.setBorderWidthTop(0.5f);
            aviso.setBorderColorTop(new Color(0xd7, 0xde, 0xdd));
            aviso.setPaddingTop(5);
            pie.addCell(aviso);

            PdfPCell pagina = new PdfPCell(new Phrase(
                    "Página " + writer.getPageNumber(), legal));
            pagina.setBorder(Rectangle.TOP);
            pagina.setBorderWidthTop(0.5f);
            pagina.setBorderColorTop(new Color(0xd7, 0xde, 0xdd));
            pagina.setPaddingTop(5);
            pagina.setHorizontalAlignment(Element.ALIGN_RIGHT);
            pie.addCell(pagina);

            pie.writeSelectedRows(0, -1, document.left(),
                    document.bottom() - 6, writer.getDirectContent());
        }
    }
```

CÓDIGO FUENTE

```
    private String fileName(ReportDto report, String formato) {
        return "Reporte-" + slug(report.type()) + "-"
                + clock.today().atTime(LocalTime.now().withNano(0)).format(SELLO_ARCHIVO)
                + "." + formato;
    }

    private String slug(String texto) {
        String sinTildes = Normalizer.normalize(texto, Normalizer.Form.NFD)
                .replaceAll("\\p{M}", "");
        return sinTildes.toLowerCase(Locale.ROOT)
                .replaceAll("[^a-z0-9]+", "-")
                .replaceAll("(^-|-$)", "");
    }

    private void writePdf(ReportDto report, String lab, String branch, OutputStream out) {

        Document doc = new Document(
                report.columns().size() >= 4 ? PageSize.A4.rotate() : PageSize.A4,
                36, 36, 40, 40);
        PdfWriter.getInstance(doc, out);

        doc.addTitle(report.type() + " · " + lab);
        doc.addCreator(lab);
        doc.open();

        Font fuenteMarca = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 16,
FONDO_CABECERA);
        Font fuenteTitulo = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 11,
FONDO_CABECERA);
        Font fuenteDato = FontFactory.getFont(FontFactory.HELVETICA, 8, Color.DARK_GRAY);
        Font fuentePie = FontFactory.getFont(FontFactory.HELVETICA_OBLIQUE, 7, Color.GRAY);

        Paragraph marca = new Paragraph(lab.toUpperCase(Locale.ROOT), fuenteMarca);
        marca.setSpacingAfter(2);
        doc.add(marca);

        Paragraph sub = new Paragraph(branch, fuenteDato);
        sub.setSpacingAfter(12);
        doc.add(sub);

        Paragraph titulo = new Paragraph(report.type(), fuenteTitulo);
        titulo.setSpacingAfter(2);
        doc.add(titulo);

        Paragraph meta = new Paragraph("Período: " + report.range()
                + " · del " + fecha(report.from()) + " al " + fecha(report.to())
                + " · generado el " + fecha(clock.todayIso()), fuenteDato);
        meta.setSpacingAfter(10);
        doc.add(meta);

        doc.add(tabla(report));

        Paragraph pie = new Paragraph(
                "Documento generado por el Sistema de Gestión de Laboratorio Clínico · "
                        + "No constituye un documento clínico ni un diagnóstico médico.",
                fuentePie);
        pie.setSpacingBefore(12);
        doc.add(pie);

        doc.close();
    }

    private PdfPTable tabla(ReportDto report) {
        PdfPTable tabla = new PdfPTable(report.columns().size());
        tabla.setWidthPercentage(100);

        Font fuenteEncabezado = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 7,
TURQUESA);
        Font fuenteCelda = FontFactory.getFont(FontFactory.HELVETICA, 8, Color.BLACK);

        for (String columna : report.columns()) {
            PdfPCell celda = new PdfPCell(new Phrase(columna.toUpperCase(Locale.ROOT),
                    fuenteEncabezado));
```

CÓDIGO FUENTE

```
            celda.setBackgroundColor(FONDO_CABECERA);
            celda.setPadding(6);
            celda.setBorderWidth(0);
            tabla.addCell(celda);
        }
        tabla.setHeaderRows(1);

        boolean alterna = false;
        for (List<String> fila : report.rows()) {
            for (String valor : fila) {
                PdfPCell celda = new PdfPCell(new Phrase(valor, fuenteCelda));
                celda.setPadding(5);
                celda.setBorderWidth(0.3f);
                celda.setBorderColor(new Color(0xdd, 0xe5, 0xe4));
                if (alterna) {
                    celda.setBackgroundColor(GRIS_SUAVE);
                }
                tabla.addCell(celda);
            }
            alterna = !alterna;
        }

        if (report.rows().isEmpty()) {
            PdfPCell vacia = new PdfPCell(new Phrase("Sin datos en el período
seleccionado.",
                    FontFactory.getFont(FontFactory.HELVETICA_OBLIQUE, 8, Color.GRAY)));
            vacia.setColspan(report.columns().size());
            vacia.setPadding(12);
            vacia.setHorizontalAlignment(Element.ALIGN_CENTER);
            tabla.addCell(vacia);
        }

        return tabla;
    }

    private String fecha(String iso) {
        return iso == null || iso.isBlank()
                ? "—"
                : java.time.LocalDate.parse(iso).format(FECHA_ES_PE);
    }

    private void writeXlsx(ReportDto report, String lab, String branch, OutputStream out)
            throws IOException {
        try (XSSFWorkbook libro = new XSSFWorkbook()) {

            String nombreHoja = report.type().length() > 31
                    ? report.type().substring(0, 31)
                    : report.type();
            Sheet hoja = libro.createSheet(nombreHoja);

            CellStyle estiloEncabezado = libro.createCellStyle();
            org.apache.poi.ss.usermodel.Font negrita = libro.createFont();
            negrita.setBold(true);
            negrita.setColor(IndexedColors.WHITE.getIndex());
            estiloEncabezado.setFont(negrita);
            estiloEncabezado.setFillForegroundColor(IndexedColors.DARK_TEAL.getIndex());
            estiloEncabezado.setFillPattern(FillPatternType.SOLID_FOREGROUND);
            estiloEncabezado.setBorderBottom(BorderStyle.THIN);
            estiloEncabezado.setAlignment(HorizontalAlignment.LEFT);

            int fila = 0;

            Row cabecera = hoja.createRow(fila++);
            cabecera.createCell(0).setCellValue(lab + " · " + branch);

            Row metadatos = hoja.createRow(fila++);
            metadatos.createCell(0).setCellValue(report.type()
                    + " · Período: " + report.range()
                    + " · del " + fecha(report.from()) + " al " + fecha(report.to()));

            fila++;

            Row encabezados = hoja.createRow(fila++);
            for (int c = 0; c < report.columns().size(); c++) {
                var celda = encabezados.createCell(c);
                celda.setCellValue(report.columns().get(c));
```

CÓDIGO FUENTE

```
                celda.setCellStyle(estiloEncabezado);
            }

            for (List<String> datos : report.rows()) {
                Row r = hoja.createRow(fila++);
                for (int c = 0; c < datos.size(); c++) {
                    r.createCell(c).setCellValue(datos.get(c));
                }
            }

            hoja.createRow(fila + 1).createCell(0)
                    .setCellValue("Documento generado por el Sistema de Gestión de
Laboratorio Clínico.");

            for (int c = 0; c < report.columns().size(); c++) {
                hoja.autoSizeColumn(c);
            }

            libro.write(out);
        }
    }
}
```

9. RESPALDO Y ELIMINACIÓN DE ATENCIONES

service/BackupService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/BackupService.java

Respaldo de pacientes en CSV: una fila por parámetro de resultado publicado, con datos personales, solicitud, muestra, historial de estados y resultado. Protege los campos contra fórmulas al abrirse en Excel.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Condor;
import pe.vizcacha.laboratorio.domain.Llama;
import pe.vizcacha.laboratorio.domain.Puma;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.domain.VicunaResult;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;
import pe.vizcacha.laboratorio.dto.ExportResultDto;
import pe.vizcacha.laboratorio.dto.ResultReportDto;
import pe.vizcacha.laboratorio.repo.CondorRepository;
import pe.vizcacha.laboratorio.repo.PumaRepository;
import pe.vizcacha.laboratorio.repo.VicunaRepository;


import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;


@Service
public class BackupService {


    private static final DateTimeFormatter FECHA =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
    private static final DateTimeFormatter FECHA_HORA =
DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
    private static final DateTimeFormatter SELLO = DateTimeFormatter.ofPattern("yyyy-MM-dd-
HHmm");


    static final List<String> COLUMNAS = List.of(
            "Código del paciente", "DNI", "Nombres", "Apellidos", "Fecha de nacimiento",
"Sexo",
            "Teléfono", "Correo", "Dirección", "Notas del paciente", "Fecha de registro",
            "Solicitud", "Fecha de solicitud", "Solicitud con pendientes",
            "Muestra", "Código del examen", "Examen", "Área", "Tipo de muestra",
            "Fecha de toma", "Tomada por", "Estado actual",
            "Recibida", "En análisis", "En validación", "Validado", "Publicada",
            "Parámetro", "Resultado", "Unidad", "Valores de referencia", "Indicador",
            "Observación del parámetro", "Observaciones generales");


    private final PumaRepository patients;
    private final CondorRepository requests;
    private final VicunaRepository samples;
    private final ResultReportService informes;
    private final BuhoService audit;
    private final SessionService session;
    private final DemoClock clock;


    public BackupService(PumaRepository patients, CondorRepository requests,
                         VicunaRepository samples, ResultReportService informes,
                         BuhoService audit, SessionService session, DemoClock clock) {
        this.patients = patients;
        this.requests = requests;
        this.samples = samples;
```

CÓDIGO FUENTE

```
        this.informes = informes;
        this.audit = audit;
        this.session = session;
        this.clock = clock;
    }

    public String fileName() {
        return "Respaldo-pacientes-"
                + clock.today().atTime(LocalTime.now().withNano(0)).format(SELLO) + ".csv";
    }

    @Transactional(readOnly = true)
    public byte[] csv() {
        return csv(null);
    }

    @Transactional(readOnly = true)
    public byte[] csv(java.util.Set<String> soloMuestras) {
        StringBuilder out = new StringBuilder("");
        linea(out, COLUMNAS);
        List<Condor> todasLasSolicitudes = requests.findAll();
        List<Vicuna> todasLasMuestras = samples.findAll();
        List<Puma> pacientes = new ArrayList<>(patients.findAll());
        pacientes.sort(Comparator.comparing(Puma::getId));
        for (Puma p : pacientes) {
            List<Condor> suyas = todasLasSolicitudes.stream()
                    .filter(r -> r.getPatient().getId().equals(p.getId()))
                    .sorted(Comparator.comparing(Condor::getId)).toList();
            if (soloMuestras != null) {
                todasLasMuestras.stream()
                        .filter(s -> soloMuestras.contains(s.getId())
                                && s.getPatient().getId().equals(p.getId()))
                        .sorted(Comparator.comparing(Vicuna::getId))
                        .forEach(s -> filasDeMuestra(out, p, s.getRequest(), s.getExam(),
s));
                continue;
            }
            if (suyas.isEmpty()) {
                linea(out, fila(p, null, null, null, null));
                continue;
            }
            for (Condor r : suyas) {
                for (Llama examen : r.getExams()) {
                    List<Vicuna> muestras = todasLasMuestras.stream()
                            .filter(s -> s.getRequest().getId().equals(r.getId())
                                    && s.getExam().getId().equals(examen.getId()))
                            .sorted(Comparator.comparing(Vicuna::getId)).toList();
                    if (muestras.isEmpty()) {
                        linea(out, fila(p, r, examen, null, null));
                        continue;
                    }
                    for (Vicuna s : muestras) {
                        filasDeMuestra(out, p, r, examen, s);
                    }
                }
            }
        }
        return out.toString().getBytes(StandardCharsets.UTF_8);
    }

    @Transactional
    public ExportResultDto saveTo(Path archivo) {
        byte[] contenido = csv();
        try {
            Path carpeta = archivo.toAbsolutePath().getParent();
            if (carpeta != null) {
                Files.createDirectories(carpeta);
            }
            Files.write(archivo, contenido);
        } catch (IOException e) {
            throw new BusinessException("No se pudo escribir el respaldo: " +
e.getMessage());
        }
        registrar(archivo.getFileName().toString());
        return new ExportResultDto(archivo.getFileName().toString(),
                archivo.toAbsolutePath().toString(), contenido.length);
    }

    @Transactional
    public void registrar(String nombre) {
```

CÓDIGO FUENTE

```
        audit.record(session.userName(), session.role(),
                "Respaldo de pacientes exportado", "Configuración", nombre);
    }

    private void filasDeMuestra(StringBuilder out, Puma p, Condor r, Llama examen, Vicuna
s) {
        if (s.getState() != VicunaState.PUBLICADA) {
            linea(out, fila(p, r, examen, s, null));
            return;
        }
        ResultReportDto informe = informes.build(s.getId());
        if (informe.rows().isEmpty()) {
            linea(out, fila(p, r, examen, s, null));
            return;
        }
        for (ResultReportDto.Row row : informe.rows()) {
            linea(out, fila(p, r, examen, s, row));
        }
    }

    private List<String> fila(Puma p, Condor r, Llama examen, Vicuna s, ResultReportDto.Row
row) {
        List<String> c = new ArrayList<>();
        c.add(p.getId());
        c.add(p.getDni());
        c.add(p.getFirstName());
        c.add(p.getLastName());
        c.add(fecha(p.getBirth()));
        c.add(p.getSex());
        c.add(p.getPhone());
        c.add(p.getEmail());
        c.add(p.getAddress());
        c.add(p.getNotes());
        c.add(fecha(p.getCreated()));
        c.add(r == null ? "" : r.getId());
        c.add(r == null ? "" : fecha(r.getDate()));
        c.add(r == null ? "" : r.isPending() ? "Sí" : "No");
        c.add(s == null ? (examen == null ? "" : "Sin muestra tomada") : s.getId());
        c.add(examen == null ? "" : examen.getId());
        c.add(examen == null ? "" : examen.getName());
        c.add(examen == null ? "" : examen.getCategory());
        c.add(examen == null ? "" : examen.getSampleType());
        c.add(s == null ? "" : fechaHora(s.getTakenAt()));
        c.add(s == null ? "" : s.getResponsible());
        c.add(s == null ? "" : s.getState().label());
        for (VicunaState estado : VicunaState.values()) {
            c.add(s == null ? "" : traza(s, estado));
        }
        if (row != null) {
            c.add(row.name());
            c.add(row.value());
            c.add(row.unit());
            c.add(row.reference());
            c.add(row.flag());
            c.add("—".equals(row.observation()) ? "" : row.observation());
        } else if (s != null && s.getState() != VicunaState.PUBLICADA) {
            c.add("");
            c.add("Resultado no publicado");
            c.add("");
            c.add("");
            c.add("");
            c.add("");
        } else {
            for (int i = 0; i < 6; i++) {
                c.add("");
            }
        }
        c.add(s == null || s.getState() != VicunaState.PUBLICADA ? "" : s.getNotes());
        return c;
    }

    private String traza(Vicuna s, VicunaState estado) {
        return s.getTraces().stream().filter(t -> t.getState() == estado).findFirst()
                .map(t -> fechaHora(t.getAt()) + " · " + t.getUserName())
                .orElse("");
    }

    private static String fecha(LocalDate d) {
        return d == null ? "" : d.format(FECHA);
    }
```

CÓDIGO FUENTE

```
    private static String fechaHora(LocalDateTime d) {
        return d == null ? "" : d.format(FECHA_HORA);
    }

    private static void linea(StringBuilder out, List<String> campos) {
        for (int i = 0; i < campos.size(); i++) {
            if (i > 0) {
                out.append(',');
            }
            out.append(campo(campos.get(i)));
        }
        out.append("\r\n");
    }

    static String campo(String valor) {
        String v = valor == null ? "" : valor;
        if (!v.isEmpty() && "=+@\t\r".indexOf(v.charAt(0)) >= 0
                || v.startsWith("-") && !v.matches("-\\d+([.,]\\d+)?")) {
            v = "'" + v;
        }
        return "\"" + v.replace("\"", "\"\"") + "\"";
    }
}
```

service/PurgeService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/PurgeService.java

Respaldar y eliminar: selecciona las solicitudes con todos sus exámenes publicados antes de una fecha, escribe el CSV y solo si quedó completo elimina muestras, solicitudes y los pacientes que se quedan sin solicitudes.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Condor;
import pe.vizcacha.laboratorio.domain.Puma;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.domain.VicunaTrace;
import pe.vizcacha.laboratorio.domain.enums.VicunaState;
import pe.vizcacha.laboratorio.dto.ExportResultDto;
import pe.vizcacha.laboratorio.repo.CondorRepository;
import pe.vizcacha.laboratorio.repo.PumaRepository;
import pe.vizcacha.laboratorio.repo.VicunaRepository;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

@Service
public class PurgeService {

    private static final DateTimeFormatter SELLO = DateTimeFormatter.ofPattern("yyyy-MM-dd-
HHmm");
    private static final DateTimeFormatter FECHA =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
```

CÓDIGO FUENTE

```
    private final PumaRepository patients;
    private final CondorRepository requests;
    private final VicunaRepository samples;
    private final BackupService respaldos;
    private final ExportService exportaciones;
    private final BuhoService audit;
    private final SessionService session;
    private final DemoClock clock;

    public PurgeService(PumaRepository patients, CondorRepository requests,
VicunaRepository samples,
                        BackupService respaldos, ExportService exportaciones, BuhoService
audit,
                        SessionService session, DemoClock clock) {
        this.patients = patients;
        this.requests = requests;
        this.samples = samples;
        this.respaldos = respaldos;
        this.exportaciones = exportaciones;
        this.audit = audit;
        this.session = session;
        this.clock = clock;
    }

    public record Plan(Set<String> solicitudes, Set<String> muestras, Set<String>
pacientes) {
    }

    public LocalDate limite(String antesDe) {
        if (antesDe == null || antesDe.isBlank()) {
            throw new BusinessException("Indica hasta qué fecha eliminar.");
        }
        LocalDate fecha;
        try {
            fecha = LocalDate.parse(antesDe.trim());
        } catch (RuntimeException e) {
            throw new BusinessException("La fecha no es válida.");
        }
        if (fecha.isAfter(clock.today())) {
            throw new BusinessException("Solo se elimina lo entregado hasta hoy.");
        }
        return fecha;
    }

    @Transactional(readOnly = true)
    public Plan plan(LocalDate antesDe) {
        Map<String, List<Vicuna>> porSolicitud = new HashMap<>();
        for (Vicuna s : samples.findAll()) {
            porSolicitud.computeIfAbsent(s.getRequest().getId(), k -> new
ArrayList<>()).add(s);
        }
        Set<String> solicitudes = new LinkedHashSet<>();
        Set<String> muestras = new LinkedHashSet<>();
        Map<String, Integer> restantes = new HashMap<>();
        List<Condor> todas = requests.findAll();
        for (Condor r : todas) {
            restantes.merge(r.getPatient().getId(), 1, Integer::sum);
            List<Vicuna> suyas = porSolicitud.getOrDefault(r.getId(), List.of());
            if (suyas.isEmpty() || suyas.size() < r.getExams().size()) {
                continue;
            }
            if (suyas.stream().allMatch(s -> entregadaAntes(s, antesDe))) {
                solicitudes.add(r.getId());
                suyas.forEach(s -> muestras.add(s.getId()));
            }
        }
        Set<String> pacientes = new LinkedHashSet<>();
        for (Condor r : todas) {
            if (solicitudes.contains(r.getId())) {
                restantes.merge(r.getPatient().getId(), -1, Integer::sum);
            }
        }
        for (Condor r : todas) {
            String id = r.getPatient().getId();
            if (solicitudes.contains(r.getId()) && restantes.get(id) == 0) {
                pacientes.add(id);
            }
        }
        return new Plan(solicitudes, muestras, pacientes);
```

CÓDIGO FUENTE

```
    }

    private boolean entregadaAntes(Vicuna s, LocalDate antesDe) {
        if (s.getState() != VicunaState.PUBLICADA) {
            return false;
        }
        return s.getTraces().stream()
                .filter(t -> t.getState() == VicunaState.PUBLICADA)
                .map(VicunaTrace::getAt).map(LocalDateTime::toLocalDate)
                .findFirst().map(d -> d.isBefore(antesDe)).orElse(false);
    }

    @Transactional(readOnly = true)
    public Map<String, Object> vistaPrevia(String antesDe) {
        LocalDate fecha = limite(antesDe);
        Plan plan = plan(fecha);
        return Map.of("requests", plan.solicitudes().size(), "samples",
plan.muestras().size(),
                "patients", plan.pacientes().size(), "before", fecha.toString());
    }

    @Transactional(readOnly = true)
    public Plan exigirPlan(LocalDate antesDe) {
        Plan plan = plan(antesDe);
        if (plan.solicitudes().isEmpty()) {
            throw new BusinessException("No hay solicitudes entregadas por completo antes
del "
                    + antesDe.format(FECHA) + ".");
        }
        return plan;
    }

    public String fileName(LocalDate antesDe) {
        return "Respaldo-eliminados-hasta-" + antesDe.minusDays(1) + "-"
                + clock.today().atTime(LocalTime.now().withNano(0)).format(SELLO) + ".csv";
    }

    @Transactional
    public ExportResultDto respaldarEnCarpeta(String antesDe) {
        LocalDate fecha = limite(antesDe);
        return respaldarYEliminar(fecha, exportaciones.carpeta().resolve(fileName(fecha)));
    }

    @Transactional
    public ExportResultDto respaldarYEliminar(LocalDate antesDe, Path archivo) {
        Plan plan = exigirPlan(antesDe);
        byte[] contenido = respaldos.csv(plan.muestras());
        try {
            Path carpeta = archivo.toAbsolutePath().getParent();
            if (carpeta != null) {
                Files.createDirectories(carpeta);
            }
            Files.write(archivo, contenido);
            if (Files.size(archivo) != contenido.length) {
                throw new IOException("el archivo quedó incompleto");
            }
        } catch (IOException e) {
            throw new BusinessException("No se pudo guardar el respaldo, así que no se
eliminó nada: "
                    + e.getMessage());
        }
        samples.deleteAllById(plan.muestras());
        samples.flush();
        requests.deleteAllById(plan.solicitudes());
        requests.flush();
        List<Puma> sinNada = patients.findAllById(plan.pacientes());
        patients.deleteAll(sinNada);
        patients.flush();
        String nombre = archivo.getFileName().toString();
        audit.record(session.userName(), session.role(),
                "Respaldo y eliminación de entregados: " + plan.pacientes().size() + "
pacientes, "
                        + plan.solicitudes().size() + " solicitudes, " +
plan.muestras().size()
                        + " resultados", "Configuración", nombre);
        return new ExportResultDto(nombre, archivo.toAbsolutePath().toString(),
contenido.length);
    }
}
```

10. AUDITORÍA Y RESPONSABLE

service/BuhoService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/BuhoService.java

Registro de cada acción en la auditoría.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.domain.Buho;
import pe.vizcacha.laboratorio.domain.enums.Role;
import pe.vizcacha.laboratorio.repo.BuhoRepository;

@Service
public class BuhoService {

    private final BuhoRepository audit;
    private final IdSequence ids;
    private final DemoClock clock;

    public BuhoService(BuhoRepository audit, IdSequence ids, DemoClock clock) {
        this.audit = audit;
        this.ids = ids;
        this.clock = clock;
    }

    @Transactional
    public Buho record(String userName, Role role, String action,
                       String module, String recordRef) {
        Buho entry = new Buho();
        entry.setId(ids.next("AUD-", audit.findAllIds()));
        entry.setAt(clock.timestamp());
        entry.setUserName(userName);
        entry.setRole(role);
        entry.setAction(action);
        entry.setModule(module);
        entry.setRecordRef(recordRef);

        entry.setSeq(audit.minSeq() - 1);
        return audit.save(entry);
    }
}
```

service/SessionService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/SessionService.java

Devuelve el nombre del responsable configurado, con el que se firman la auditoría y la trazabilidad.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import pe.vizcacha.laboratorio.domain.Preference;
import pe.vizcacha.laboratorio.domain.enums.Role;

@Service
public class SessionService {

    public static final String REGISTRO_LOCAL = "Registro local";

    private final PreferenceService preferences;
```

CÓDIGO FUENTE

```
    public SessionService(PreferenceService preferences) {
        this.preferences = preferences;
    }

    public String userName() {
        String nombre = preferences.get(Preference.RESPONSABLE, "");
        return nombre == null || nombre.isBlank() ? Preference.RESPONSABLE_POR_DEFECTO :
nombre.trim();
    }

    public Role role() {
        return Role.ACCESO_GENERAL;
    }
}
```

11. API REST

La interfaz se comunica con el servidor únicamente a través de estas rutas, todas en el propio equipo (localhost):

Tabla 3. *Rutas de la API*

Método	Ruta	Controlador
POST	/api/resources	AlpacaController
POST	/api/resources/{id}/movements	AlpacaController
POST	/api/requests	CondorController
GET	/api/database	DatabaseController
POST	/api/desktop/save-report	DesktopController
POST	/api/desktop/save-backup	DesktopController
POST	/api/desktop/purge	DesktopController
POST	/api/desktop/save-result/{sampleId}	DesktopController
POST	/api/desktop/print/{sampleId}	DesktopController
POST	/api/desktop/folder	DesktopController
POST	/api/inventory/count	InventoryController
POST	/api/inventory/purchase	InventoryController
POST	/api/exams	LlamaController
POST	/api/patients	PumaController
GET	/api/purge/preview	PurgeController
POST	/api/purge	PurgeController
GET	/api/reports	ReportController
POST	/api/reports/export	ReportController
POST	/api/settings	SettingsController
POST	/api/settings/export-dir	SettingsController
POST	/api/settings/appearance	SettingsController
POST	/api/settings/notifications	SettingsController
POST	/api/maintenance	TapirController
POST	/api/samples	VicunaController
POST	/api/samples/{id}/state	VicunaController
GET	/api/samples/{id}/report	VicunaController
POST	/api/samples/{id}/report/pdf	VicunaController
POST	/api/samples/{id}/results	VicunaController

service/DatabaseSnapshotService.java

Ruta: src/main/java/pe/vizcacha/laboratorio/service/DatabaseSnapshotService.java

Arma la fotografía completa de los datos que recibe la interfaz tras cada operación.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.domain.Buho;
import pe.vizcacha.laboratorio.domain.Llama;
import pe.vizcacha.laboratorio.domain.Condor;
import pe.vizcacha.laboratorio.domain.Tapir;
import pe.vizcacha.laboratorio.domain.Puma;
import pe.vizcacha.laboratorio.domain.Alpaca;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.domain.VicunaResult;
import pe.vizcacha.laboratorio.domain.VicunaTrace;
import pe.vizcacha.laboratorio.domain.Preference;
import pe.vizcacha.laboratorio.domain.Settings;
import pe.vizcacha.laboratorio.domain.enums.AlpacaKind;
import pe.vizcacha.laboratorio.dto.BuhoDto;
import pe.vizcacha.laboratorio.dto.DatabaseDto;
import pe.vizcacha.laboratorio.dto.LlamaDto;
import pe.vizcacha.laboratorio.dto.TapirDto;
import pe.vizcacha.laboratorio.dto.PumaDto;
import pe.vizcacha.laboratorio.dto.CondorDto;
import pe.vizcacha.laboratorio.dto.AlpacaDto;
import pe.vizcacha.laboratorio.dto.VicunaDto;
import pe.vizcacha.laboratorio.dto.TraceDto;
import pe.vizcacha.laboratorio.repo.BuhoRepository;
import pe.vizcacha.laboratorio.repo.LlamaRepository;
import pe.vizcacha.laboratorio.repo.CondorRepository;
import pe.vizcacha.laboratorio.repo.TapirRepository;
import pe.vizcacha.laboratorio.repo.PumaRepository;
import pe.vizcacha.laboratorio.repo.AlpacaRepository;
import pe.vizcacha.laboratorio.repo.VicunaRepository;
import pe.vizcacha.laboratorio.repo.SettingsRepository;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@Service
public class DatabaseSnapshotService {

    private static final DateTimeFormatter ISO_SEGUNDOS =
            DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss");

    private static final String SUFIJO_OBSERVACION = "_obs";

    private final PumaRepository patients;
    private final LlamaRepository exams;
    private final AlpacaRepository resources;
    private final CondorRepository requests;
    private final VicunaRepository samples;
    private final TapirRepository maintenance;
    private final BuhoRepository audit;
    private final SettingsRepository settings;
    private final AvailabilityService availability;
    private final AlpacaStatusService statuses;
    private final AlertService alerts;
    private final ResultFormatCatalog formats;
    private final ExportService exports;
    private final PreferenceService preferences;
    private final SessionService session;

    public DatabaseSnapshotService(PumaRepository patients, LlamaRepository exams,
                                   AlpacaRepository resources, CondorRepository requests,
                                   VicunaRepository samples, TapirRepository maintenance,
                                   BuhoRepository audit,
                                   SettingsRepository settings,
                                   AvailabilityService availability,
                                   AlpacaStatusService statuses, AlertService alerts,
                                   ResultFormatCatalog formats,
                                   ExportService exports, PreferenceService preferences,
                                   SessionService session) {
```

CÓDIGO FUENTE

```
        this.session = session;
        this.patients = patients;
        this.exams = exams;
        this.resources = resources;
        this.requests = requests;
        this.samples = samples;
        this.maintenance = maintenance;
        this.audit = audit;
        this.settings = settings;
        this.availability = availability;
        this.statuses = statuses;
        this.alerts = alerts;
        this.formats = formats;
        this.exports = exports;
        this.preferences = preferences;
    }

    @Transactional(readOnly = true)
    public DatabaseDto snapshot() {
        Settings config = settings.findById(Settings.SINGLETON_ID).orElse(null);

        return new DatabaseDto(

patients.findAll().stream().sorted(byId(Puma::getId)).map(this::toDto).toList(),

exams.findAll().stream().sorted(byId(Llama::getId)).map(this::toDto).toList(),
                resourcesInSeedOrder(),

requests.findAll().stream().sorted(byId(Condor::getId)).map(this::toDto).toList(),

samples.findAll().stream().sorted(byId(Vicuna::getId)).map(this::toDto).toList(),

maintenance.findAll().stream().sorted(byId(Tapir::getId)).map(this::toDto).toList(),
                audit.findAllByOrderBySeqAsc().stream().map(this::toDto).toList(),
                alerts.alerts(),
                formatsByLabel(),
                config == null ? SettingsService.LAB_POR_DEFECTO : config.getLab(),
                config == null ? "Laboratorio Central" : config.getBranch(),
                config == null || config.isNotifications(),
                Boolean.getBoolean("laboratorio.ventana"),
                exports.carpeta().toAbsolutePath().toString(),
                preferences.get(Preference.APARIENCIA, Preference.APARIENCIA_POR_DEFECTO),
                session.userName());
    }

    private java.util.Map<String, List<pe.vizcacha.laboratorio.service.LlamaParam>>
formatsByLabel() {
        java.util.Map<String, List<pe.vizcacha.laboratorio.service.LlamaParam>> porEtiqueta
=
                new java.util.LinkedHashMap<>();
        for (pe.vizcacha.laboratorio.domain.enums.ResultFormat f
                : pe.vizcacha.laboratorio.domain.enums.ResultFormat.values()) {
            porEtiqueta.put(f.label(), formats.params(f));
        }
        return porEtiqueta;
    }

    private <T> Comparator<T> byId(java.util.function.Function<T, String> id) {
        return Comparator.comparing(id);
    }

    private List<AlpacaDto> resourcesInSeedOrder() {
        Comparator<Alpaca> orden = Comparator
                .comparingInt((Alpaca r) -> switch (r.getKind()) {
                    case EQUIPOS -> 0;
                    case REACTIVOS -> 1;
                    case MATERIALES -> 2;
                })
                .thenComparing(Alpaca::getId);

        return resources.findAll().stream().sorted(orden).map(this::toDto).toList();
    }

    private PumaDto toDto(Puma p) {
        return new PumaDto(p.getId(), p.getDni(), p.getFirstName(), p.getLastName(),
                iso(p.getBirth()), p.getSex(), nz(p.getPhone()), nz(p.getEmail()),
                nz(p.getAddress()), nz(p.getNotes()), iso(p.getCreated()));
    }
```

CÓDIGO FUENTE

```
    private LlamaDto toDto(Llama e) {
        List<String> motivos = availability.reasons(e);
        return new LlamaDto(e.getId(), e.getName(), e.getCategory(), e.getPrice(),
                e.getTimeLabel(), e.getSampleType(), e.getVolume(), nz(e.getPrep()),
                e.getEquipment().getId(), e.getReagent().getId(), e.getMaterial().getId(),
                e.isEnabled(), e.getFormat().label(),
                motivos.isEmpty(), motivos);
    }

    private AlpacaDto toDto(Alpaca r) {
        return new AlpacaDto(r.getId(), r.getName(), r.getKind().label(),
                nz(r.getBrand()), nz(r.getModel()), nz(r.getSerial()), r.getArea(),
                r.getStatus() == null ? "" : r.getStatus().label(),
                r.getQty(), r.getMinQty(), r.getUnit(), nz(r.getLot()),
                iso(r.getExpires()), iso(r.getLastMaint()), iso(r.getNextMaint()),
                nz(r.getResponsible()), statuses.derived(r));
    }

    private CondorDto toDto(Condor r) {
        return new CondorDto(r.getId(), r.getPatient().getId(),
                r.getExams().stream().map(Llama::getId).toList(),
                iso(r.getDate()), r.isPending());
    }

    private VicunaDto toDto(Vicuna s) {
        return new VicunaDto(s.getId(), s.getPatient().getId(), s.getRequest().getId(),
                s.getExam().getId(), iso(s.getTakenAt()), s.getState().label(),
                s.getResponsible(), flattenResults(s), nz(s.getNotes()),
                s.getTraces().stream().map(this::toDto).toList());
    }

    private Map<String, String> flattenResults(Vicuna s) {
        Map<String, String> plano = new LinkedHashMap<>();
        for (VicunaResult r : s.getResults()) {
            plano.put(r.getParamName(), nz(r.getValue()));
            if (r.getObservation() != null && !r.getObservation().isBlank()) {
                plano.put(r.getParamName() + SUFIJO_OBSERVACION, r.getObservation());
            }
        }
        return plano;
    }

    private TraceDto toDto(VicunaTrace t) {
        return new TraceDto(t.getState().label(), iso(t.getAt()), t.getUserName());
    }

    private TapirDto toDto(Tapir m) {
        return new TapirDto(m.getId(), m.getEquipment().getId(), m.getType(),
                iso(m.getStartDate()), iso(m.getEndDate()), iso(m.getActualDate()),
                m.getResponsible(), nz(m.getProvider()), nz(m.getDescription()),
                m.getState().label());
    }

    private BuhoDto toDto(Buho a) {
        return new BuhoDto(a.getId(), iso(a.getAt()), a.getUserName(), a.getRole().label(),
                a.getAction(), a.getModule(), a.getRecordRef());
    }

    private String iso(LocalDate d) {
        return d == null ? "" : d.toString();
    }

    private String iso(LocalDateTime d) {
        return d == null ? "" : d.format(ISO_SEGUNDOS);
    }

    private String nz(String s) {
        return s == null ? "" : s;
    }
}
```

web/VicunaController.java

Ruta: src/main/java/pe/vizcacha/laboratorio/web/VicunaController.java

Controlador de muestras: registro, cambio de estado, resultados e informe.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.web;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.vizcacha.laboratorio.domain.Vicuna;
import pe.vizcacha.laboratorio.dto.DatabaseDto;
import pe.vizcacha.laboratorio.dto.ExportResultDto;
import pe.vizcacha.laboratorio.dto.NewVicunaForm;
import pe.vizcacha.laboratorio.dto.ResultForm;
import pe.vizcacha.laboratorio.dto.ResultReportDto;
import pe.vizcacha.laboratorio.dto.StateForm;
import pe.vizcacha.laboratorio.service.DatabaseSnapshotService;
import pe.vizcacha.laboratorio.service.ExportService;
import pe.vizcacha.laboratorio.service.ResultReportService;
import pe.vizcacha.laboratorio.service.ResultService;
import pe.vizcacha.laboratorio.service.VicunaService;

@RestController
@RequestMapping("/api/samples")
public class VicunaController {

    private final VicunaService samples;
    private final ResultService results;
    private final DatabaseSnapshotService snapshots;
    private final ResultReportService reports;
    private final ExportService exports;

    public VicunaController(VicunaService samples, ResultService results,
                            DatabaseSnapshotService snapshots,
                            ResultReportService reports, ExportService exports) {
        this.samples = samples;
        this.results = results;
        this.snapshots = snapshots;
        this.reports = reports;
        this.exports = exports;
    }

    @PostMapping
    public ResponseEntity<DatabaseDto> register(@RequestBody NewVicunaForm form) {
        Vicuna registrada = samples.register(form);
        return ApiResponses.withRecord(snapshots.snapshot(), registrada.getId());
    }

    @PostMapping("/{id}/state")
    public ResponseEntity<DatabaseDto> state(@PathVariable String id,
                                             @RequestBody StateForm form) {
        Vicuna movida = samples.transition(id, form.state());
        return ApiResponses.withRecord(snapshots.snapshot(), movida.getId());
    }

    @GetMapping("/{id}/report")
    public ResultReportDto report(@PathVariable String id) {
        return reports.build(id);
    }

    @PostMapping("/{id}/report/pdf")
    public ExportResultDto reportPdf(@PathVariable String id) {
        return exports.exportResult(reports.build(id));
    }

    @GetMapping("/{id}/report/pdf/file")
    public ResponseEntity<byte[]> reportPdfFile(@PathVariable String id) {
        ResultReportDto informe = reports.build(id);
        return ResponseEntity.ok()
                .header(org.springframework.http.HttpHeaders.CONTENT_DISPOSITION,
```

CÓDIGO FUENTE

```
                        org.springframework.http.ContentDisposition.attachment()
                                .filename(exports.resultFileName(informe))
                                .build().toString())
                .contentType(org.springframework.http.MediaType.APPLICATION_PDF)
                .body(exports.resultPdf(informe));
    }

    @PostMapping("/{id}/results")
    public ResponseEntity<DatabaseDto> results(@PathVariable String id,
                                               @RequestBody ResultForm form) {
        Vicuna guardada = results.save(id, form);
        return ApiResponses.withRecord(snapshots.snapshot(), guardada.getId());
    }
}
```

web/DesktopController.java

Ruta: src/main/java/pe/vizcacha/laboratorio/web/DesktopController.java

Funciones exclusivas de la ventana de escritorio: diálogos del sistema para elegir carpeta y guardar informes, reportes y respaldos, e impresión con el visor de PDF.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.web;

import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.vizcacha.laboratorio.VentanaVizcacha;
import pe.vizcacha.laboratorio.domain.Preference;
import pe.vizcacha.laboratorio.dto.ExportForm;
import pe.vizcacha.laboratorio.dto.ExportResultDto;
import pe.vizcacha.laboratorio.dto.ReportDto;
import pe.vizcacha.laboratorio.dto.ResultReportDto;
import pe.vizcacha.laboratorio.service.BusinessException;
import pe.vizcacha.laboratorio.service.ExportService;
import pe.vizcacha.laboratorio.service.PreferenceService;
import pe.vizcacha.laboratorio.service.ResultReportService;


import java.awt.Desktop;
import java.io.File;
import java.io.IOException;
import java.util.Map;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;

@RestController
@RequestMapping("/api/desktop")
public class DesktopController {

    private final ResultReportService informes;
    private final ExportService exportaciones;
    private final PreferenceService preferencias;
    private final ReportController reportes;
    private final pe.vizcacha.laboratorio.service.BackupService respaldos;
    private final pe.vizcacha.laboratorio.service.PurgeService depuracion;

    public DesktopController(ResultReportService informes, ExportService exportaciones,
                             PreferenceService preferencias, ReportController reportes,
                             pe.vizcacha.laboratorio.service.BackupService respaldos,
                             pe.vizcacha.laboratorio.service.PurgeService depuracion) {
        this.depuracion = depuracion;
        this.respaldos = respaldos;
        this.informes = informes;
        this.exportaciones = exportaciones;
        this.preferencias = preferencias;
        this.reportes = reportes;
    }

    @PostMapping("/save-report")
```

CÓDIGO FUENTE

```
    public Map<String, Object> guardarReporte(@RequestBody ExportForm form) {
        soloEnLaVentana();
        String formato = exportaciones.formato(form.format());
        ReportDto reporte = reportes.reporte(form);
        boolean pdf = ExportService.PDF.equals(formato);
        String ruta = pedirArchivo(
                pdf ? "Guardar el reporte en PDF" : "Guardar el reporte en Excel",
                exportaciones.reportFileName(reporte, formato),
                pdf ? "Documento PDF" : "Libro de Excel", "." + formato,
                carpetaExistente(exportaciones.carpeta().toAbsolutePath().toString()));
        if (ruta.isEmpty()) {
            return Map.of("path", "", "cancelled", true);
        }
        java.nio.file.Path destino = java.nio.file.Paths.get(ruta);
        ExportResultDto guardado = exportaciones.exportTo(reporte, formato,
                reportes.lab(), reportes.sede(), destino);
        preferencias.set(Preference.CARPETA_EXPORTACIONES,
                destino.toAbsolutePath().getParent().toString());
        return Map.of("path", guardado.path(), "fileName", guardado.fileName(),
                "cancelled", false);
    }

    @PostMapping("/save-backup")
    public Map<String, Object> guardarRespaldo() {
        soloEnLaVentana();
        String ruta = pedirArchivo("Guardar el respaldo de pacientes",
respaldos.fileName(),
                "Archivo CSV", ".csv",
                carpetaExistente(exportaciones.carpeta().toAbsolutePath().toString()));
        if (ruta.isEmpty()) {
            return Map.of("path", "", "cancelled", true);
        }
        ExportResultDto guardado = respaldos.saveTo(java.nio.file.Paths.get(ruta));
        return Map.of("path", guardado.path(), "fileName", guardado.fileName(),
                "cancelled", false);
    }

    @PostMapping("/purge")
    public Map<String, Object> respaldarYEliminar(@RequestBody Map<String, String> cuerpo)
{
        soloEnLaVentana();
        java.time.LocalDate fecha = depuracion.limite(cuerpo.get("before"));
        depuracion.exigirPlan(fecha);
        String ruta = pedirArchivo("Guardar el respaldo antes de eliminar",
depuracion.fileName(fecha),
                "Archivo CSV", ".csv",
                carpetaExistente(exportaciones.carpeta().toAbsolutePath().toString()));
        if (ruta.isEmpty()) {
            return Map.of("path", "", "cancelled", true);
        }
        ExportResultDto guardado = depuracion.respaldarYEliminar(fecha,
java.nio.file.Paths.get(ruta));
        return Map.of("path", guardado.path(), "fileName", guardado.fileName(),
                "cancelled", false);
    }

    private void soloEnLaVentana() {
        if (!Boolean.getBoolean("laboratorio.ventana")) {
            throw new BusinessException("El selector de carpetas solo está "
                    + "disponible en la aplicación de escritorio.");
        }
    }

    private String pedirArchivo(String titulo, String nombre, String tipo, String
extension,
                                File inicial) {
        CompletableFuture<String> elegido = new CompletableFuture<>();
        javafx.application.Platform.runLater(() -> {
            try {
                javafx.stage.FileChooser selector = new javafx.stage.FileChooser();
                selector.setTitle(titulo);
                selector.setInitialFileName(nombre);
                selector.getExtensionFilters().add(
                        new javafx.stage.FileChooser.ExtensionFilter(tipo, "*" +
extension));
                if (inicial != null) {
                    selector.setInitialDirectory(inicial);
                }
                File archivo = selector.showSaveDialog(VentanaVizcacha.principal());
                elegido.complete(archivo == null ? "" : archivo.getAbsolutePath());
```

CÓDIGO FUENTE

```
            } catch (RuntimeException e) {
                elegido.completeExceptionally(e);
            }
        });
        String ruta;
        try {
            ruta = elegido.get(5, TimeUnit.MINUTES);
        } catch (Exception e) {
            throw new BusinessException("No se pudo abrir la ventana para elegir la
carpeta.");
        }
        if (!ruta.isEmpty() && !
ruta.toLowerCase(java.util.Locale.ROOT).endsWith(extension)) {
            ruta = ruta + extension;
        }
        return ruta;
    }

    private File carpetaExistente(String... candidatas) {
        for (String candidata : candidatas) {
            if (candidata == null || candidata.isBlank()) {
                continue;
            }
            File carpeta = new File(candidata);
            if (carpeta.isDirectory()) {
                return carpeta;
            }
        }
        File casa = new File(System.getProperty("user.home"));
        return casa.isDirectory() ? casa : null;
    }

    @PostMapping("/save-result/{sampleId}")
    public Map<String, Object> guardarResultado(@PathVariable String sampleId) {
        soloEnLaVentana();
        ResultReportDto informe = informes.build(sampleId);
        String ruta = pedirArchivo("Guardar el informe de resultados",
                exportaciones.resultFileName(informe), "Documento PDF", ".pdf",
                carpetaExistente(preferencias.get(Preference.CARPETA_RESULTADOS, ""),
                        exportaciones.carpeta().toAbsolutePath().toString()));
        if (ruta.isEmpty()) {
            return Map.of("path", "", "cancelled", true);
        }
        java.nio.file.Path destino = java.nio.file.Paths.get(ruta);
        ExportResultDto guardado = exportaciones.exportResultTo(informe, destino);
        preferencias.set(Preference.CARPETA_RESULTADOS,
                destino.toAbsolutePath().getParent().toString());
        return Map.of("path", guardado.path(), "fileName", guardado.fileName(),
                "cancelled", false);
    }

    @PostMapping("/print/{sampleId}")
    public ExportResultDto imprimir(@PathVariable String sampleId) {
        ExportResultDto archivo = exportaciones.exportResult(informes.build(sampleId));
        abrir(new File(archivo.path()));
        return archivo;
    }

    @PostMapping("/folder")
    public Map<String, String> elegirCarpeta() {
        if (!Boolean.getBoolean("laboratorio.ventana")) {
            throw new BusinessException("El selector de carpetas solo está "
                    + "disponible en la aplicación de escritorio. Escribe la ruta.");
        }
        CompletableFuture<String> elegida = new CompletableFuture<>();
        javafx.application.Platform.runLater(() -> {
            try {
                javafx.stage.DirectoryChooser selector = new
javafx.stage.DirectoryChooser();
                selector.setTitle("Carpeta para los PDF y los Excel");
                File inicial =
carpetaExistente(exportaciones.carpeta().toAbsolutePath().toString());
                if (inicial != null) {
                    selector.setInitialDirectory(inicial);
                }
                File carpeta = selector.showDialog(VentanaVizcacha.principal());
                elegida.complete(carpeta == null ? "" : carpeta.getAbsolutePath());
            } catch (RuntimeException e) {
                elegida.completeExceptionally(e);
            }
```

CÓDIGO FUENTE

```
        });
        try {
            return Map.of("path", elegida.get(3, TimeUnit.MINUTES));
        } catch (Exception e) {
            throw new BusinessException("No se pudo abrir el selector de carpetas. "
                    + "Escribe la ruta a mano.");
        }
    }

    private void abrir(File archivo) {
        if (!Desktop.isDesktopSupported()
                || !Desktop.getDesktop().isSupported(Desktop.Action.OPEN)) {
            throw new BusinessException("El sistema no permite abrir el archivo. "
                    + "El informe quedó guardado en " + archivo.getAbsolutePath());
        }
        try {
            Desktop.getDesktop().open(archivo);
        } catch (IOException e) {
            throw new BusinessException("No hay un visor de PDF configurado. "
                    + "El informe quedó guardado en " + archivo.getAbsolutePath());
        }
    }
}
```

web/ApiExceptionHandler.java

Ruta: src/main/java/pe/vizcacha/laboratorio/web/ApiExceptionHandler.java

Convierte los errores en respuestas HTTP: 422 con el mensaje para las reglas de negocio.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio.web;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import pe.vizcacha.laboratorio.service.BusinessException;

import java.time.format.DateTimeParseException;
import java.util.Map;

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<Map<String, String>> business(BusinessException e) {
        return ResponseEntity.unprocessableEntity().body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, String>> illegal(IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(DateTimeParseException.class)
    public ResponseEntity<Map<String, String>> fecha(DateTimeParseException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("error", "Fecha con formato no válido: " +
e.getParsedString()));
    }
}
```

12. INTERFAZ DE USUARIO

frontend/src/main.tsx

Ruta: frontend/src/main.tsx

Punto de entrada: aplica el tema guardado y monta la aplicación.

CÓDIGO FUENTE

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './index.css'


try {
  const tema = localStorage.getItem('laboratorio-appearance-v1')
  if (tema === 'blanco' || tema === 'gris' || tema === 'negro') {
    document.documentElement.dataset.theme = tema
  }
} catch {
  document.documentElement.dataset.theme = 'gris'
}

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

frontend/src/api.ts

Ruta: frontend/src/api.ts

Cliente de la API. Toda modificación devuelve los datos completos actualizados; los errores 422 se muestran con su mensaje literal.

CÓDIGO FUENTE

```
import type { Database, ResultReport } from "./data"

export class ApiError extends Error {}

export type Mutation = { db: Database; recordId: string }

const CABECERA_ID = "X-Record-Id"

async function parse(response: Response) {
  const texto = await response.text()
  if (!response.ok) {
    let mensaje = `Error ${response.status}`
    try {
      mensaje = JSON.parse(texto).error || mensaje
    } catch {
    }
    if (response.status >= 500) {
      mensaje =
        "Ocurrió un error inesperado y no se guardó nada. Vuelve a intentarlo; si se
repite, el detalle está en datos/laboratorio.log."
    }
    throw new ApiError(mensaje)
  }
  return texto ? JSON.parse(texto) : null
}

async function get<T>(url: string): Promise<T> {
  return parse(await fetch(url))
}

async function post(url: string, body?: unknown): Promise<Mutation> {
  const response = await fetch(url, {
```

CÓDIGO FUENTE

```
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body ?? {}),
  })
  const db = await parse(response)
  return { db, recordId: response.headers.get(CABECERA_ID) || "" }
}

export type Report = {
  type: string
  range: string
  from: string
  to: string
  columns: string[]
  rows: string[][]
}

export type ExportResult = {
  fileName: string
  path: string
  bytes: number
}

export const api = {

  database: () => get<Database>("/api/database"),

  report: (type: string, range: string, from?: string, to?: string) => {
    const params = new URLSearchParams({ type, range })
    if (from) params.set("from", from)
    if (to) params.set("to", to)
    return get<Report>(`/api/reports?${params}`)
  },

  savePatient: (form: Record<string, unknown>) => post("/api/patients", form),

  createRequest: (patientId: string, examIds: string[], allowPending: boolean) =>
    post("/api/requests", { patientId, examIds, allowPending }),

  registerSample: (
    requestId: string,
    examId: string,
    takenAt: string,
    responsible: string,
  ) => post("/api/samples", { requestId, examId, takenAt, responsible }),

  sampleState: (id: string, state: string) =>
    post(`/api/samples/${id}/state`, { state }),

  saveResults: (
    id: string,
    values: Record<string, string>,
    notes: string,
    submit: boolean,
  ) => post(`/api/samples/${id}/results`, { values, notes, submit }),

  saveExam: (form: Record<string, unknown>) => post("/api/exams", form),
  saveResource: (form: Record<string, unknown>) => post("/api/resources", form),

  move: (id: string, direction: "IN" | "OUT", qty: number, notes: string) =>
    post(`/api/resources/${id}/movements`, { direction, qty, notes }),

  inventoryCount: (form: Record<string, unknown>) =>
    post("/api/inventory/count", form),
  purchase: (form: Record<string, unknown>) =>
    post("/api/inventory/purchase", form),

  saveMaintenance: (form: Record<string, unknown>) =>
    post("/api/maintenance", form),

  saveSettings: (lab: string, branch: string, responsible: string) =>
    post("/api/settings", { lab, branch, responsible }),
  setNotifications: (enabled: boolean) =>
    post("/api/settings/notifications", { enabled }),
```

CÓDIGO FUENTE

```
  setExportDir: (path: string) => post("/api/settings/export-dir", { path }),
  setAppearance: (theme: string) =>
    post("/api/settings/appearance", { theme }),

  resultReport: (sampleId: string) =>
    get<ResultReport>(`/api/samples/${sampleId}/report`),

  printOnDesktop: (sampleId: string) =>
    fetch(`/api/desktop/print/${sampleId}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: "{}",
    }).then(parse) as Promise<ExportResult>,

  saveResultOnDesktop: (sampleId: string) =>
    fetch(`/api/desktop/save-result/${sampleId}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: "{}",
    }).then(parse) as Promise<{
      path: string
      fileName?: string
      cancelled: boolean
    }>,

  saveReportOnDesktop: (
    type: string,
    range: string,
    format: "pdf" | "xlsx",
    from?: string,
    to?: string,
  ) =>
    fetch("/api/desktop/save-report", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ type, range, format, from, to }),
    }).then(parse) as Promise<{
      path: string
      fileName?: string
      cancelled: boolean
    }>,

  saveBackupOnDesktop: () =>
    fetch("/api/desktop/save-backup", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: "{}",
    }).then(parse) as Promise<{
      path: string
      fileName?: string
      cancelled: boolean
    }>,

  backupCsvUrl: () => "/api/backup/patients.csv",

  purgePreview: (before: string) =>
    get<{ requests: number; samples: number; patients: number; before: string }>(
      `/api/purge/preview?before=${before}`,
    ),

  purgeToFolder: (before: string) =>
    fetch("/api/purge", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ before }),
    }).then(parse) as Promise<ExportResult>,

  purgeOnDesktop: (before: string) =>
    fetch("/api/desktop/purge", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ before }),
    }).then(parse) as Promise<{
      path: string
      fileName?: string
      cancelled: boolean
    }>,
```

CÓDIGO FUENTE

```
  resultPdfFileUrl: (sampleId: string) =>
    `/api/samples/${sampleId}/report/pdf/file`,

  chooseFolder: () =>
    fetch("/api/desktop/folder", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: "{}",
    }).then(parse) as Promise<{ path: string }>,

  exportReport: (
    type: string,
    range: string,
    format: "pdf" | "xlsx",
    from?: string,
    to?: string,
  ) =>
    fetch("/api/reports/export", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ type, range, format, from, to }),
    }).then(parse) as Promise<ExportResult>,
}
```

App.tsx — etapas del flujo

Ruta: frontend/src/App.tsx (fragmento)

Definición de las siete etapas del flujo de atención: nombre, sección, estado y lo que falta para avanzar.

CÓDIGO FUENTE

```
const ETAPAS = [
  {
    id: "atender",
    nombre: "Por atender",
    etiqueta: "Por atender",
    falta: "Crear la solicitud",
    seccion: "Pacientes",
    tono: "neutral",
    unidad: "pacientes",
  },
  {
    id: "muestra",
    nombre: "Toma de muestra",
    etiqueta: "Toma de muestra",
    falta: "Registrar la muestra",
    seccion: "Solicitudes",
    tono: "pendiente",
    unidad: "solicitudes",
  },
  {
    id: "RECIBIDA",
    nombre: "Por analizar",
    etiqueta: "Por analizar",
    falta: "Iniciar el análisis",
    seccion: "Muestras",
    tono: "recibida",
    unidad: "muestras",
  },
  {
    id: "EN ANÁLISIS",
    nombre: "Por ingresar",
    etiqueta: "Por ingresar resultados",
    falta: "Ingresar los resultados",
    seccion: "Resultados",
    tono: "análisis",
    unidad: "muestras",
  },
  {
    id: "EN VALIDACIÓN",
    nombre: "Por validar",
    etiqueta: "Por validar",
```

CÓDIGO FUENTE

```
    falta: "Revisar y validar",
    seccion: "Resultados",
    tono: "validación",
    unidad: "resultados",
  },
  {
    id: "VALIDADO",
    nombre: "Por publicar",
    etiqueta: "Por publicar",
    falta: "Publicar el resultado",
    seccion: "Resultados",
    tono: "validado",
    unidad: "resultados",
  },
  {
    id: "PUBLICADA",
    nombre: "Entregados",
    etiqueta: "Entregado",
    falta: "Atención completa",
    seccion: "Resultados",
    tono: "publicada",
    unidad: "hoy",
  },
]
const indiceDeEstado = (estado: string) =>
  ETAPAS.findIndex((e) => e.id === estado)
const PESTANAS_RESULTADOS = ETAPAS.slice(3).map((e) => e.nombre)
```

App.tsx — etapa de cada registro y barra del flujo

Ruta: frontend/src/App.tsx (fragmento)

Funciones que ubican a cada paciente y solicitud en su etapa y dibujan la barra del flujo.

CÓDIGO FUENTE

```
  const fechaDeEstado = (s: Vicuna, estado = s.state) =>
    s.history.find((h) => h.state === estado)?.date || s.date
  const faltanMuestras = (r: Condor) =>
    r.exams.filter(
      (id) => !db.samples.some((s) => s.request === r.id && s.exam === id),
    )
  const etapaDeSolicitud = (r: Condor) => {
    if (faltanMuestras(r).length) return 1
    const indices = db.samples
      .filter((s) => s.request === r.id)
      .map((s) => indiceDeEstado(s.state))
    return indices.length ? Math.min(...indices) : 6
  }
  const etapaDePaciente = (id: string) => {
    const reqs = db.requests.filter((r) => r.patient === id)
    return reqs.length ? Math.min(...reqs.map(etapaDeSolicitud)) : 0
  }
  const sinSolicitud = (id: string) => !db.requests.some((r) => r.patient === id)
  const publicadosHoy = db.samples.filter(
    (s) =>
      s.state === "PUBLICADA" && fechaDeEstado(s, "PUBLICADA").startsWith(TODAY),
  )
  const conteoDeEtapa = (i: number) =>
    i === 0
      ? db.patients.filter((p) => sinSolicitud(p.id)).length
      : i === 1
        ? db.requests.filter((r) => faltanMuestras(r).length).length
        : i === 6
          ? publicadosHoy.length
          : statusCount(ETAPAS[i].id)
  const etapaBadge = (i: number) => (
    <Badge value={ETAPAS[i].tono}>{ETAPAS[i].etiqueta}</Badge>
  )
  function Flujo(actual?: string, pestana?: string) {
    return (
      <div className="flujo" aria-label="Flujo del laboratorio">
        {ETAPAS.map((e, i) => {
          const n = conteoDeEtapa(i)
          return (
            <button
```

CÓDIGO FUENTE

```
              key={e.id}
              className={`flujo-paso ${actual === e.seccion && (!pestana || pestana ===
e.nombre) ? "en-seccion" : ""} ${n ? "" : "vacio"}`}
              onClick={() => go(e.seccion, e.nombre)}
              title={`${e.nombre}: ${e.falta}`}
            >
              <span className="flujo-numero">{i + 1}</span>
              <span className="flujo-texto">
                <strong>{dosDigitos(n)}</strong>
                <small>{e.nombre}</small>
                <em>
                  {e.seccion}
                  {i === 6 ? " · hoy" : ""}
                </em>
              </span>
            </button>
          )
        })}
      </div>
    )
  }
```

App.tsx — pantalla de Resultados

Ruta: frontend/src/App.tsx (fragmento)

Pantalla de Resultados con sus cuatro pestañas.

CÓDIGO FUENTE

```
  function ResultsPage() {
    const pestana = PESTANAS_RESULTADOS.includes(filter) ? filter : "Por ingresar"
    const estado = ETAPAS.find((e) => e.nombre === pestana)!.id
    const entregados = pestana === "Entregados"
    const items = filteredSamples
      .filter((s) => s.state === estado)
      .sort((a, b) =>
        entregados
          ? fechaDeEstado(b).localeCompare(fechaDeEstado(a))
          : fechaDeEstado(a).localeCompare(fechaDeEstado(b)),
      )
    const pag = bloque("resultados", items)
    const siguiente: Record<string, string> = {
      "Por ingresar": "al enviarlos a validación pasan a Por validar",
      "Por validar": "al validarlos pasan a Por publicar",
      "Por publicar": "al publicarlos quedan entregados al paciente",
      Entregados: "del más reciente al más antiguo",
    }
    return (
      <>
        {Flujo("Resultados", pestana)}
        <Tabs
          items={PESTANAS_RESULTADOS}
          counts={Object.fromEntries(
            ETAPAS.slice(3).map((e) => [e.nombre, statusCount(e.id)]),
          )}
          active={pestana}
          onChange={setFilter}
        />
        <Notice>
          Los borradores y los resultados en validación{" "}
          <strong>no se consideran publicados</strong>. Cada cambio queda en la
          auditoría.
        </Notice>
        <Panel>
          {SampleFilters(false)}
          <DataTable
            columns={[
              "Código de muestra",
              "Paciente / DNI",
              "Examen",
              "Fecha de toma",
              entregados ? "Publicado" : "En esta etapa desde",
              "Estado",
              "Responsable",
              "Acciones",
```

CÓDIGO FUENTE

```
            ]}
            rows={pag.visibles.map((s) => [
              sampleLink(s),
              patientCell(s.patient),
              exam(s.exam).name,
              dateLabel(s.date),
              <div className="stacked">
                <span>{dateLabel(fechaDeEstado(s))}</span>
                <small>{timeLabel(fechaDeEstado(s))}</small>
              </div>,
              <Badge value={s.state} />,
              s.responsible,
              resultActions(s),
            ])}
          />
          <Paginador {...pag} />
          <div className="table-footer">
            {items.length
              ? `${items.length} ${entregados ? "resultados entregados" : "resultados"} · $
{siguiente[pestana]}`
              : `No hay resultados en ${pestana}.`}
          </div>
        </Panel>
      </>
    )
  }
```

13. PRUEBAS AUTOMÁTICAS

El proyecto tiene 134 pruebas automáticas que se ejecutan con mvn test. Cubren el esquema de la base, los datos iniciales, la disponibilidad, el flujo completo de atención, los resultados y el informe, el inventario y las compras, los mantenimientos, las exportaciones, el respaldo y la eliminación de atenciones.

Tabla 4. *Pruebas automáticas*

Clase de prueba	Qué verifica
SchemaTest	Tablas e índices de la base.
SnapshotTest	La fotografía de datos que recibe la interfaz.
RulesTest	Disponibilidad, estados derivados y alertas.
FlowTest	El flujo completo: paciente, solicitud, muestra, resultados, validación y publicación.
FormatsTest	Formatos de parámetros de resultados.
ResultReportTest	El informe de resultados y sus firmas.
AlpacasTest	Recursos, movimientos, inventario y mantenimiento.
PurchaseTest	Compras con varias líneas.
AdminTest	Catálogo, configuración y reportes.
ExportTest	Exportaciones a PDF y Excel.
PreferencesTest	Carpetas, tema y responsable.
BackupTest	Respaldo de pacientes en CSV.
PurgeTest	Respaldar y eliminar atenciones antiguas.
SeedTimesTest, DemoClockTest	Fechas de los datos de ejemplo y reloj del sistema.

test/PurgeTest.java

Ruta: src/test/java/pe/vizcacha/laboratorio/PurgeTest.java

Ejemplo de prueba: comprueba que se respalda primero, que solo se elimina lo entregado, que lo que está en curso se conserva y que, si no se puede guardar el respaldo, no se elimina nada.

CÓDIGO FUENTE

```
package pe.vizcacha.laboratorio;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.laboratorio.config.DemoClock;
import pe.vizcacha.laboratorio.service.PurgeService;

import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
```

CÓDIGO FUENTE

```
import java.util.HashSet;
import java.util.Set;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class PurgeTest {

    @Autowired private MockMvc mvc;
    @Autowired private ObjectMapper json;
    @Autowired private PurgeService depuracion;
    @Autowired private DemoClock clock;

    private JsonNode get(String url) throws Exception {
        return
json.readTree(mvc.perform(org.springframework.test.web.servlet.request.MockMvcRequestBuilde
rs.get(url))
                .andExpect(status().isOk()).andReturn().getResponse().getContentAsByteArray
());
    }

    private Set<String> ids(JsonNode lista) {
        Set<String> ids = new HashSet<>();
        lista.forEach(n -> ids.add(n.get("id").asText()));
        return ids;
    }

    @Test
    void respaldaPrimeroYEliminaSoloLoEntregado() throws Exception {
        JsonNode antes = get("/api/database");
        PurgeService.Plan plan = depuracion.plan(clock.today());
        assertThat(plan.solicitudes()).isNotEmpty();
        for (JsonNode s : antes.get("samples")) {
            if (plan.muestras().contains(s.get("id").asText())) {
                assertThat(s.get("state").asText()).isEqualTo("PUBLICADA");
            }
        }

        Path destino = Files.createTempDirectory("depuracion").resolve("respaldo.csv");
        depuracion.respaldarYEliminar(clock.today(), destino);

        String csv = Files.readString(destino, StandardCharsets.UTF_8);
        plan.muestras().forEach(id -> assertThat(csv).contains("\"" + id + "\""));
        for (JsonNode p : antes.get("patients")) {
            if (plan.pacientes().contains(p.get("id").asText())) {
                assertThat(csv).contains("\"" + p.get("dni").asText() + "\"");
            }
        }

        JsonNode despues = get("/api/database");
        Set<String> muestrasDespues = ids(despues.get("samples"));
        Set<String> solicitudesDespues = ids(despues.get("requests"));
        plan.muestras().forEach(id -> assertThat(muestrasDespues).doesNotContain(id));
        plan.solicitudes().forEach(id ->
assertThat(solicitudesDespues).doesNotContain(id));
        Set<String> pacientesDespues = ids(despues.get("patients"));
        plan.pacientes().forEach(id -> assertThat(pacientesDespues).doesNotContain(id));
        assertThat(despues.get("samples").size())
                .isEqualTo(antes.get("samples").size() - plan.muestras().size());

assertThat(despues.get("resources").size()).isEqualTo(antes.get("resources").size());
        assertThat(despues.get("exams").size()).isEqualTo(antes.get("exams").size());
        assertThat(despues.get("audit").size()).isEqualTo(antes.get("audit").size() + 1);
        assertThat(despues.get("audit").get(0).get("action").asText())
                .startsWith("Respaldo y eliminación de entregados");
    }

    @Test
    void loQueEstaEnCursoYSusPacientesSeQuedan() throws Exception {
        JsonNode antes = get("/api/database");
        Set<String> conAlgoEnCurso = new HashSet<>();
```

CÓDIGO FUENTE

```
        Set<String> enCurso = new HashSet<>();
        for (JsonNode s : antes.get("samples")) {
            if (!s.get("state").asText().equals("PUBLICADA")) {
                enCurso.add(s.get("id").asText());
                conAlgoEnCurso.add(s.get("patient").asText());
            }
        }
        assertThat(enCurso).isNotEmpty();
        depuracion.respaldarYEliminar(clock.today(),
Files.createTempDirectory("b").resolve("b.csv"));
        JsonNode despues = get("/api/database");
        assertThat(ids(despues.get("samples"))).containsAll(enCurso);
        assertThat(ids(despues.get("patients"))).containsAll(conAlgoEnCurso);
    }

    @Test
    void siNoSePuedeGuardarElRespaldoNoSeEliminaNada() throws Exception {
        int antes = get("/api/database").get("samples").size();
        assertThatThrownBy(() -> depuracion.respaldarYEliminar(clock.today(),
                Path.of("/proc/no-se-puede/respaldo.csv")))
                .hasMessageStartingWith("No se pudo guardar el respaldo, así que no se
eliminó nada");
        assertThat(get("/api/database").get("samples").size()).isEqualTo(antes);
    }

    @Test
    void repetirAvisaQueNoQuedaNada() throws Exception {
        depuracion.respaldarYEliminar(clock.today(),
Files.createTempDirectory("c").resolve("c.csv"));
        mvc.perform(post("/api/purge").contentType(MediaType.APPLICATION_JSON)
                        .content("{\"before\":\"" + clock.today() + "\"}"))
                .andExpect(status().isUnprocessableEntity())
                .andExpect(jsonPath("$.error").value(org.hamcrest.Matchers.startsWith(
                        "No hay solicitudes entregadas por completo antes del")));
    }

    @Test
    void noSeEliminaHaciaElFuturoNiSinVentanaElSelector() throws Exception {
        mvc.perform(post("/api/purge").contentType(MediaType.APPLICATION_JSON)
                        .content("{\"before\":\"" + clock.today().plusDays(1) + "\"}"))
                .andExpect(status().isUnprocessableEntity())
                .andExpect(jsonPath("$.error").value("Solo se elimina lo entregado hasta
hoy."));
        mvc.perform(post("/api/desktop/purge").contentType(MediaType.APPLICATION_JSON)
                        .content("{\"before\":\"" + clock.today() + "\"}"))
                .andExpect(status().isUnprocessableEntity());
    }
}
```

14. EMPAQUETADO

packaging/empaquetar-linux.sh

Ruta: packaging/empaquetar-linux.sh

Genera el paquete de Linux con jpackage: compila, recorta el Java con jlink a los módulos necesarios y crea el lanzador nativo.

CÓDIGO FUENTE

```
#!/usr/bin/env bash

set -euo pipefail

RAIZ="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CACHE="$RAIZ/packaging/.cache"
DESTINO="$RAIZ/dist/linux"
JDK="${JAVA_HOME:-$HOME/.jdks/jdk-21.0.12.1+1}"
VERSION="1.0.0"

MODULOS="java.base,java.desktop,java.instrument,java.logging,java.management"
MODULOS="$MODULOS,java.naming,java.net.http,java.prefs,java.scripting"
MODULOS="$MODULOS,java.security.jgss,java.sql,java.transaction.xa,java.xml"
MODULOS="$MODULOS,java.xml.crypto,jdk.crypto.ec,jdk.unsupported,jdk.zipfs"

paso() { printf '\n\033[1;36m==> %s\033[0m\n' "$1"; }

paso "Compilando (nativas de Linux)"
cd "$RAIZ"
JAVA_HOME="$JDK" mvn -q -B clean package -DskipTests

JAR="$RAIZ/target/laboratorio-clinico-$VERSION.jar"
[ -f "$JAR" ] || { echo "No se genero el jar"; exit 1; }

if [ ! -x "$CACHE/runtime-linux/bin/java" ]; then
  paso "Recortando el runtime con jlink"
  rm -rf "$CACHE/runtime-linux"
  "$JDK/bin/jlink" --module-path "$JDK/jmods" --add-modules "$MODULOS" \
    --output "$CACHE/runtime-linux" --strip-debug --no-header-files --no-man-pages
fi

paso "Desplegando el jar"
rm -rf "$CACHE/app-linux"
"$JDK/bin/java" -Djarmode=tools -jar "$JAR" extract --destination "$CACHE/app-linux"
>/dev/null
mv "$CACHE/app-linux/laboratorio-clinico-$VERSION.jar" "$CACHE/app-linux/laboratorio-
clinico.jar"

OPCIONES=(
  --name Laboratorio
  --app-version "$VERSION"
  --vendor "Laboratorio clinico"
  --description "Sistema de gestion de laboratorio clinico"
  --copyright "Laboratorio clinico"
  --input "$CACHE/app-linux"
  --main-jar laboratorio-clinico.jar
  --main-class pe.vizcacha.laboratorio.DesktopLauncher
  --runtime-image "$CACHE/runtime-linux"
  --icon "$RAIZ/packaging/Laboratorio-256.png"
  --java-options -XX:TieredStopAtLevel=1
  --java-options -XX:+UseSerialGC
  --java-options -Xms64m
  --java-options -Dfile.encoding=UTF-8
)

paso "Generando la carpeta ejecutable"
rm -rf "$DESTINO"
mkdir -p "$DESTINO"
"$JDK/bin/jpackage" --type app-image --dest "$DESTINO" "${OPCIONES[@]}"
cp "$RAIZ/packaging/LEEME-linux.txt" "$DESTINO/Laboratorio/" 2>/dev/null || true

if [ "${1:-}" = "rpm" ]; then
  paso "Generando el paquete RPM"
  "$JDK/bin/jpackage" --type rpm --dest "$DESTINO" "${OPCIONES[@]}" \
```

CÓDIGO FUENTE

```
    --linux-shortcut \
    --linux-menu-group "Office;MedicalSoftware" \
    --linux-package-name laboratorio-clinico \
    --linux-app-category Office
fi

paso "Listo"
printf '  %s\n' "$DESTINO/Laboratorio"
du -sh "$DESTINO/Laboratorio" | awk '{print "  tamaño: "$1}'
ls "$DESTINO"/*.rpm 2>/dev/null | sed 's/^/  /' || true
printf '\n  Para ejecutarla:  %s/Laboratorio/bin/Laboratorio\n' "$DESTINO"
```

packaging/launch4j.xml

Ruta: packaging/launch4j.xml

Configuración del ejecutable Laboratorio.exe: Java incluido en runtime, imagen de carga, icono y datos de versión.

CÓDIGO FUENTE

```
<launch4jConfig>
  <dontWrapJar>true</dontWrapJar>
  <headerType>gui</headerType>
  <jar>app\laboratorio-clinico.jar</jar>
  <outfile>../dist/Laboratorio/Laboratorio.exe</outfile>
  <errTitle>Laboratorio clinico</errTitle>
  <chdir>.</chdir>
  <priority>normal</priority>
  <downloadUrl>https://adoptium.net/</downloadUrl>
  <stayAlive>false</stayAlive>
  <restartOnCrash>false</restartOnCrash>
  <icon>Laboratorio.ico</icon>

  <singleInstance>
    <mutexName>pe.vizcacha.laboratorio</mutexName>
    <windowTitle>Laboratorio cl</windowTitle>
  </singleInstance>

  <jre>
    <path>runtime</path>
    <requires64Bit>true</requires64Bit>
    <opt>-XX:TieredStopAtLevel=1</opt>
    <opt>-XX:+UseSerialGC</opt>
    <opt>-Xms64m</opt>
    <opt>-Dfile.encoding=UTF-8</opt>
    <opt>-Dsun.stdout.encoding=UTF-8</opt>
    <opt>-Dsun.stderr.encoding=UTF-8</opt>
  </jre>

  <splash>
    <file>splash.bmp</file>
    <waitForWindow>true</waitForWindow>
    <timeout>60</timeout>
    <timeoutErr>false</timeoutErr>
  </splash>

  <versionInfo>
    <fileVersion>1.0.0.0</fileVersion>
    <txtFileVersion>1.0.0</txtFileVersion>
    <fileDescription>Sistema de gestion de laboratorio clinico</fileDescription>
    <copyright>Laboratorio clinico</copyright>
    <productVersion>1.0.0.0</productVersion>
    <txtProductVersion>1.0.0</txtProductVersion>
    <productName>Laboratorio clinico</productName>
    <companyName>Laboratorio clinico</companyName>
    <internalName>Laboratorio</internalName>
    <originalFilename>Laboratorio.exe</originalFilename>
  </versionInfo>
</launch4jConfig>
```

15. ÍNDICE COMPLETO DE ARCHIVOS

Tabla 5. *Archivos del proyecto*

Archivo	Función
.gitignore	Archivos que no se suben al repositorio.
README.md	Presentación del proyecto en GitHub.
frontend/index.html	Página base de la interfaz.
frontend/package-lock.json	Versiones exactas de las dependencias de la interfaz.
frontend/package.json	Dependencias de la interfaz (npm).
frontend/src/App.tsx	Aplicación de interfaz: todas las pantallas, formularios y ventanas.
frontend/src/api.ts	Cliente de la API REST.
frontend/src/components.tsx	Componentes visuales reutilizables (botones, paneles, tablas, iconos, ventanas).
frontend/src/data.ts	Tipos de datos y funciones de fecha y formato.
frontend/src/desplegable.tsx	Listas desplegables propias, iguales en la ventana y en el navegador.
frontend/src/fecha.tsx	Campos de fecha y de fecha y hora con calendario.
frontend/src/index.css	Estilos y los tres temas de color.
frontend/src/main.tsx	Punto de entrada de la interfaz.
frontend/src/vite-env.d.ts	Tipos del entorno de Vite.
frontend/tsconfig.json	Configuración de TypeScript.
frontend/vite.config.ts	Configuración de Vite.
packaging/LEEME-linux.txt	Instrucciones incluidas en el paquete de Linux.
packaging/LEEME.txt	Instrucciones incluidas en el paquete de Windows.
packaging/Laboratorio-256.png	Icono del programa (Linux).
packaging/Laboratorio.ico	Icono del programa (Windows).
packaging/empaquetar-linux.sh	Genera el paquete de Linux.
packaging/empaquetar-windows.sh	Genera el paquete de Windows (Laboratorio.exe).
packaging/launch4j.xml	Configuración del ejecutable de Windows.
packaging/splash.bmp	Imagen de carga del ejecutable de Windows.
pom.xml	Proyecto Maven: dependencias, compilación y empaquetado.
java/DesktopLauncher.java	Punto de entrada del ejecutable.
java/VentanaVizcacha.java	Ventana de escritorio con JavaFX.
java/VizcachaApplication.java	Aplicación Spring Boot y carpetas de datos.
java/config/DemoClock.java	Reloj del sistema: fecha y hora que usa todo el programa.

Archivo	Función
java/config/SeedLoader.java	Carga inicial de datos al arrancar.
java/domain/Alpaca.java	Entidad persistente: recurso (equipo, reactivo o material).
java/domain/Buho.java	Entidad persistente: entrada de auditoría.
java/domain/Condor.java	Entidad persistente: solicitud.
java/domain/Llama.java	Entidad persistente: examen del catálogo.
java/domain/Preference.java	Entidad persistente: preferencia clave-valor.
java/domain/Puma.java	Entidad persistente: paciente.
java/domain/Settings.java	Entidad persistente: configuración del laboratorio.
java/domain/Tapir.java	Entidad persistente: mantenimiento.
java/domain/Vicuna.java	Entidad persistente: muestra.
java/domain/VicunaResult.java	Entidad persistente: valor de un parámetro de resultado.
java/domain/VicunaTrace.java	Entidad persistente: paso de trazabilidad de una muestra.
java/domain/Zorro.java	Entidad persistente: usuario (tabla conservada, sin uso).
java/domain/converters/ AlpacaKindConverter.java	Conversión entre el tipo AlpacaKind y su texto en la base de datos.
java/domain/converters/ EquipmentStatusConverter.java	Conversión entre el tipo EquipmentStatus y su texto en la base de datos.
java/domain/converters/ LabeledEnumConverter.java	Conversión entre el tipo LabeledEnum y su texto en la base de datos.
java/domain/converters/ LocalDateConverter.java	Conversión entre el tipo LocalDate y su texto en la base de datos.
java/domain/converters/ LocalDateTimeConverter.java	Conversión entre el tipo LocalDateTime y su texto en la base de datos.
java/domain/converters/ ResultFormatConverter.java	Conversión entre el tipo ResultFormat y su texto en la base de datos.
java/domain/converters/RoleConverter.java	Conversión entre el tipo Role y su texto en la base de datos.
java/domain/converters/ TapirStateConverter.java	Conversión entre el tipo TapirState y su texto en la base de datos.
java/domain/converters/ VicunaStateConverter.java	Conversión entre el tipo VicunaState y su texto en la base de datos.
java/domain/enums/AlpacaKind.java	Enumeración AlpacaKind: valores posibles y su etiqueta.
java/domain/enums/EquipmentStatus.java	Enumeración EquipmentStatus: valores posibles y su etiqueta.

Archivo	Función
java/domain/enums/LabeledEnum.java	Enumeración LabeledEnum: valores posibles y su etiqueta.
java/domain/enums/ResultFormat.java	Enumeración ResultFormat: valores posibles y su etiqueta.
java/domain/enums/Role.java	Enumeración Role: valores posibles y su etiqueta.
java/domain/enums/TapirState.java	Enumeración TapirState: valores posibles y su etiqueta.
java/domain/enums/VicunaState.java	Enumeración VicunaState: valores posibles y su etiqueta.
java/dto/AlertDto.java	Datos que la API envía a la interfaz.
java/dto/AlpacaDto.java	Datos que la API envía a la interfaz.
java/dto/AlpacaForm.java	Datos que recibe la API desde un formulario.
java/dto/AppearanceForm.java	Datos que recibe la API desde un formulario.
java/dto/BuhoDto.java	Datos que la API envía a la interfaz.
java/dto/CondorDto.java	Datos que la API envía a la interfaz.
java/dto/DatabaseDto.java	Datos que la API envía a la interfaz.
java/dto/ExportDirForm.java	Datos que recibe la API desde un formulario.
java/dto/ExportForm.java	Datos que recibe la API desde un formulario.
java/dto/ExportResultDto.java	Datos que la API envía a la interfaz.
java/dto/InventoryCountForm.java	Datos que recibe la API desde un formulario.
java/dto/LlamaDto.java	Datos que la API envía a la interfaz.
java/dto/LlamaForm.java	Datos que recibe la API desde un formulario.
java/dto/MovementForm.java	Datos que recibe la API desde un formulario.
java/dto/NewCondorForm.java	Datos que recibe la API desde un formulario.
java/dto/NewVicunaForm.java	Datos que recibe la API desde un formulario.
java/dto/NotificationsForm.java	Datos que recibe la API desde un formulario.
java/dto/PumaDto.java	Datos que la API envía a la interfaz.
java/dto/PumaForm.java	Datos que recibe la API desde un formulario.
java/dto/PurchaseForm.java	Datos que recibe la API desde un formulario.
java/dto/ReportDto.java	Datos que la API envía a la interfaz.
java/dto/ResultForm.java	Datos que recibe la API desde un formulario.
java/dto/ResultReportDto.java	Datos que la API envía a la interfaz.
java/dto/SettingsForm.java	Datos que recibe la API desde un formulario.
java/dto/StateForm.java	Datos que recibe la API desde un formulario.
java/dto/TapirDto.java	Datos que la API envía a la interfaz.
java/dto/TapirForm.java	Datos que recibe la API desde un formulario.

Archivo	Función
java/dto/TraceDto.java	Datos que la API envía a la interfaz.
java/dto/VicunaDto.java	Datos que la API envía a la interfaz.
java/dto/package-info.java	Descripción del paquete de datos de la API.
java/repo/AlpacaRepository.java	Acceso a la base de datos: recurso (equipo, reactivo o material).
java/repo/BuhoRepository.java	Acceso a la base de datos: entrada de auditoría.
java/repo/CondorRepository.java	Acceso a la base de datos: solicitud.
java/repo/LlamaRepository.java	Acceso a la base de datos: examen del catálogo.
java/repo/PreferenceRepository.java	Acceso a la base de datos: preferencia clave-valor.
java/repo/PumaRepository.java	Acceso a la base de datos: paciente.
java/repo/SettingsRepository.java	Acceso a la base de datos: configuración del laboratorio.
java/repo/TapirRepository.java	Acceso a la base de datos: mantenimiento.
java/repo/VicunaRepository.java	Acceso a la base de datos: muestra.
java/repo/ZorroRepository.java	Acceso a la base de datos: usuario (tabla conservada, sin uso).
java/service/AlertService.java	Calcula las alertas operativas: stock bajo, vencimientos, equipos en mantenimiento, resultados y muestras pendientes.
java/service/AlpacaService.java	Alta, edición y movimientos de entrada y salida de recursos.
java/service/AlpacaStatusService.java	Calcula el estado derivado de un recurso: Disponible, Stock bajo, Agotado o Vencido.
java/service/AvailabilityService.java	Decide si un examen está disponible según su equipo, reactivo y material.
java/service/BackupService.java	Genera el respaldo de pacientes en CSV.
java/service/BuhoService.java	Registra las entradas de la auditoría.
java/service/BusinessException.java	Error de regla de negocio que la API devuelve como 422 con su mensaje.
java/service/CondorService.java	Creación de solicitudes y cálculo de su estado pendiente.
java/service/DatabaseSnapshotService.java	Arma la fotografía completa de los datos que recibe la interfaz.
java/service/ExportService.java	Genera los PDF y Excel de reportes y el PDF del informe de resultados.
java/service/IdSequence.java	Genera los códigos correlativos (PAC-, SOL-, MUE-...).
java/service/InventoryCountService.java	Inventario físico: reemplaza existencias por el conteo.
java/service/LlamaParam.java	Definición de un parámetro de resultado (unidad, intervalo, opciones).

Archivo	Función
java/service/LlamaService.java	Alta y edición de exámenes del catálogo.
java/service/Numbers.java	Formato de números para los valores de resultados.
java/service/PreferenceService.java	Lectura y escritura de preferencias clave-valor.
java/service/PumaService.java	Registro y edición de pacientes.
java/service/PurchaseService.java	Registro de compras con varias líneas.
java/service/PurgeService.java	Respalda en CSV y elimina las atenciones antiguas ya entregadas.
java/service/ReportService.java	Calcula los seis reportes por período.
java/service/ResultFormatCatalog.java	Plantillas de parámetros de cada formato de resultado.
java/service/ResultReportService.java	Arma el informe de resultados de una muestra publicada.
java/service/ResultService.java	Guarda los valores de resultados como borrador o los envía a validación.
java/service/SeedService.java	Datos de ejemplo usados por las pruebas automáticas.
java/service/SessionService.java	Nombre del responsable con el que se firma cada acción.
java/service/SettingsService.java	Configuración general, carpeta de exportaciones y apariencia.
java/service/TapirService.java	Mantenimientos y su efecto en el estado de los equipos.
java/service/VicunaService.java	Registro de muestras y cambios de estado del flujo.
java/web/AlpacaController.java	Controlador REST: Alpaca.
java/web/ApiExceptionHandler.java	Convierte los errores en respuestas HTTP con su mensaje.
java/web/ApiResponses.java	Respuestas comunes de la API.
java/web/BackupController.java	Controlador REST: Backup.
java/web/CondorController.java	Controlador REST: Condor.
java/web/DatabaseController.java	Controlador REST: Database.
java/web/DesktopController.java	Controlador REST: Desktop.
java/web/InventoryController.java	Controlador REST: Inventory.
java/web/LlamaController.java	Controlador REST: Llama.
java/web/PumaController.java	Controlador REST: Puma.
java/web/PurgeController.java	Controlador REST: Purge.
java/web/ReportController.java	Controlador REST: Report.
java/web/SettingsController.java	Controlador REST: Settings.
java/web/TapirController.java	Controlador REST: Tapir.
java/web/VicunaController.java	Controlador REST: Vicuna.

Archivo	Función
src/main/resources/application.yml	Configuración del servidor, la base de datos y el registro.
src/main/resources/db/schema.sql	Estructura de la base de datos.
test/AdminTest.java	Pruebas automáticas: Admin.
test/AlpacasTest.java	Pruebas automáticas: Alpacas.
test/BackupTest.java	Pruebas automáticas: Backup.
test/DemoClockTest.java	Pruebas automáticas: DemoClock.
test/ExportTest.java	Pruebas automáticas: Export.
test/FlowTest.java	Pruebas automáticas: Flow.
test/FormatsTest.java	Pruebas automáticas: Formats.
test/PreferencesTest.java	Pruebas automáticas: Preferences.
test/PurchaseTest.java	Pruebas automáticas: Purchase.
test/PurgeTest.java	Pruebas automáticas: Purge.
test/ResultReportTest.java	Pruebas automáticas: ResultReport.
test/RulesTest.java	Pruebas automáticas: Rules.
test/SchemaTest.java	Pruebas automáticas: Schema.
test/SeedTimesTest.java	Pruebas automáticas: SeedTimes.
test/SnapshotTest.java	Pruebas automáticas: Snapshot.
src/test/resources/application.properties	Configuración de las pruebas automáticas.

En la tabla, java/ abrevia src/main/java/pe/vizcacha/laboratorio/ y test/ abrevia src/test/java/pe/vizcacha/laboratorio/.